ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR
Membre de HONORIS UNITED UNIVERSITIES

EMSI RECHERCHE INNOVATION

PROJET DE FIN D'ANNÉE

4e année en Ingénierie Informatique et Réseaux

Token Efficient

Système multi-agents coopératif fondé sur les LLM et économe en tokens

Token-Efficient Cooperative LLM-based Multi-Agent Systems

Compression de contexte, validation Python
et évaluation expérimentale sur modèles locaux

RÉALISÉ PAR

Abdelkader Elassali

ENCADRÉ PAR

M. Hicham Bouchtib

ORGANISME D'ACCUEIL

EMSI Recherche & Innovation

Année universitaire

2025/2026

Remerciements

Je remercie **EMSI Recherche & Innovation**, cadre d'accueil de ce projet de fin d'année, ainsi que **M. Hicham Bouchtib**, mon encadrant. Ce travail a été l'occasion d'étudier une question concrète de l'intelligence artificielle : comment réduire les ressources consommées par des agents coopératifs tout en contrôlant la fiabilité des informations qu'ils échangent.

J'adresse également mes remerciements à l'équipe pédagogique de l'École Marocaine des Sciences de l'Ingénieur pour la formation dans laquelle s'inscrit ce projet, et aux membres du jury pour l'examen de ce mémoire.

La réalisation de Token Efficient m'a permis de mettre en relation le développement logiciel, l'expérimentation et l'analyse critique des résultats. Au-delà de la mise en œuvre du prototype, cette démarche m'a appris à distinguer une amélioration observée d'une promesse générale, et à relier chaque conclusion aux données qui permettent de la vérifier.

Résumé

Les workflows utilisant plusieurs agents de modèles de langage peuvent retransmettre les mêmes faits, contraintes et rapports à plusieurs étapes. Cette répétition augmente la quantité de tokens traités. Réduire ce contexte sans contrôler les informations conservées peut toutefois introduire des omissions ou laisser circuler des affirmations contradictoires.

Ce projet étudie une solution hybride appliquée à la sélection de devis logistiques fictifs. Un registre protégé conserve la source structurée et les conditions fournisseur. Un compresseur déterministe représente ces informations et leurs contrôles sous une forme compacte. Python calcule l'admissibilité des devis et remplace certains appels spécialisés ; LangGraph orchestre les étapes et conserve l'historique brut. Llama 3 est exécuté localement via Ollama. Les rapports structurés sont contrôlés avant transmission et la sélection finale doit correspondre au calcul de référence.

L'évaluation distingue trois branches : contexte complet, compression seule, et compression accompagnée d'outils Python. Six expériences historiques v12, réparties sur trois sources et deux répétitions, ont observé des réductions totales comprises entre 43,06 % et 57,25 %. Les dix-huit décisions et les six audits de contexte associés ont satisfait leurs contrôles. Dans le protocole v13, un essai court a mesuré 42,48 % d'économie et un cas exigeant le transport réfrigéré 56,46 %. Deux cas incomplets ou non pris en charge ont été arrêtés avant inférence.

Ces résultats portent sur un petit ensemble de scénarios, et les économies totales incluent l'effet du routage ainsi que la variation des sorties. Ils ne garantissent ni une économie minimale sur toute entrée ni la compréhension de toute condition rédigée en langage naturel.

Mots-clés : compression de contexte ; agents LLM ; LangGraph ; Ollama ; validation Python ; tokens ; reproductibilité.

Abstract

Multi-agent language-model workflows can repeatedly transmit the same source facts, constraints and intermediate reports. This increases token consumption, while uncontrolled summarization can remove important qualifications or allow contradictory claims to propagate.

This project studies a hybrid solution for selecting fictional shipping quotes. A protected record retains the structured source and supplier terms. A deterministic compressor produces a compact source representation and a concise constraint audit. Python computes eligibility and replaces selected specialist model calls. LangGraph orchestrates the workflow and preserves raw history, while Llama 3 runs locally through Ollama. Structured reports are checked before forwarding, and the final selection must agree with the reference calculation.

The evaluation compares full context, compression alone, and compression with Python tools. Six historical v12 experiments across three sources and two repetitions observed total token reductions from 43.06% to 57.25%. All eighteen decisions and six context audits passed their scoped checks. In v13, a short case saved 42.48% and a refrigeration case saved 56.46%; two unsupported or incomplete cases stopped before inference.

These small-sample results demonstrate a measured workflow improvement, not a universal compression guarantee. Total savings include routing and output variation. Source preservation and agreement with Python checks do not certify general natural-language understanding or independent model reasoning.

Keywords: context compression; LLM agents; LangGraph; Ollama; Python validation; tokens; reproducibility.

Table des matières

Remerciements i

Résumé ii

Abstract iii

Liste des abréviations ix

Introduction générale 1

1 Cadre du stage et du projet 2

1.1 Présentation du cadre d'accueil . . . 2

1.2 Positionnement du sujet de stage . . . 2

1.3 Mission et livrables . . . 2

1.4 Démarche de réalisation . . . 3

1.5 Périmètre et responsabilités . . . 3

1.6 Conclusion . . . 3

2 Cadrage et choix techniques 4

2.1 Problématique . . . 4

2.2 Notions utiles : modèle, agent et état . . . 4

2.3 Analyse des approches possibles . . . 5

2.4 Objectifs et critères d'acceptation . . . 5

2.5 Besoins fonctionnels . . . 6

2.6 Besoins non fonctionnels . . . 6

2.7 Environnement technique . . . 6

2.8 Conclusion . . . 7

3 Conception et modélisation 8

3.1 Architecture générale . . . 8

3.2 Cas d'utilisation . . . 9

3.3 Modèle des données et état 9
3.4 Calcul de référence en Python 10
3.5 Compression déterministe et routage 11
3.5.1 Ce que le compresseur conserve 11
3.5.2 Exemple de représentation compacte 11
3.5.3 Deux chemins de traitement 12
3.6 Orchestration LangGraph 12
3.6.1 Responsabilités des rôles 13
3.7 Barrières de validation 14
3.7.1 Revue des exigences avant inférence 14
3.7.2 Contrôle des rapports et des contradictions 15
3.7.3 Validation de la décision 15
3.8 Séquence d'une comparaison 16
3.9 Conclusion 16

4 Réalisation et présentation de l'application 17
4.1 Organisation du code 17
4.2 API et interface 18
4.3 Présentation du tableau de bord 18
4.3.1 Une interface organisée autour de l'expérience 18
4.3.2 Choisir un exemple et préparer une comparaison 18
4.4 Consulter la décision et les rapports des agents 20
4.5 Comparer les coûts et examiner les contrôles 22
4.5.1 Les trois branches visibles 22
4.5.2 Liste de vérification de la décision 22
4.5.3 Tokens par rôle et export de la trace 24
4.6 Construire un scénario personnalisé 24
4.6.1 Saisie de l'envoi et des offres 24
4.6.2 Revue des exigences et responsabilité de la saisie 25
4.7 Comprendre les statuts affichés 26
4.8 Apport de l'interface à la démarche de recherche 26
4.9 Conclusion 26

5 Évaluation expérimentale et discussion 27
5.1 Protocole de mesure 27

5.1.1 Trois branches et une ingestion partagée 27
5.1.2 Coûts et pourcentages 28
5.1.3 Qualité et couverture 28
5.2 Cas d'étude court : décision et tokens 29
5.2.1 D'où proviennent les 2 185 tokens économisés ? 30
5.3 Résultats répétés du protocole historique v12 31
5.4 Vérification des exigences en v13 32
5.5 Tests logiciels et contrôles d'interface 33
5.6 Discussion et limites 33
5.6.1 Ce que les résultats démontrent 33
5.6.2 Ce qu'elles ne démontrent pas 33
5.6.3 Améliorations évaluables 34
5.7 Conclusion 34

Conclusion générale et perspectives **35**

A Traçabilité des mesures **36**

B Reproduction et démonstration **38**
B.1 Relevé de l'environnement lors de la revue finale 38
B.2 Préparation de l'environnement 38
B.3 Tests et vérification 38
B.4 Reproduire la campagne historique 39

C Dictionnaire des résultats et statuts **40**
C.1 Lecture des issues 41
C.2 Conservation des preuves 41

Bibliographie et sources techniques **42**

Table des figures

3.1 Architecture logique des composants observés dans le dépôt. 8
3.2 Vue simplifiée des cas d'utilisation réellement exposés. 9
3.3 Modèle des données : seules ProtectedFacts et QuoteFacts sont des classes métier figées ; les autres blocs sont des structures de données. 10
3.4 Dépendances du graphe et exécution hybride. Dans les deux branches de contrôle, les six rôles restent des appels LLM. 13
3.5 Contrôle des entrées et du résultat ; les erreurs d'exécution constituent un état d'échec distinct. 14
3.6 Séquence simplifiée. Les étapes 5 et 6 se répètent selon les rôles ; les branches partagent l'ingestion. 16

4.1 Vue générale du tableau de bord 19
4.2 Lecture de la décision dans l'application 21
4.3 Comparaison des trois branches 22
4.4 Extrait des contrôles détaillés de la décision 23
4.5 Répartition des tokens par rôle 24
4.6 Préparation d'un scénario personnalisé 25

5.1 Plan de comparaison : la branche B sépare la compression de l'effet additionnel des outils Python. 27
5.2 Tokens du cas court v13 : la troisième barre mesure l'optimisation combinée, pas le compresseur seul. 30
5.3 Décomposition du total de 5 144 tokens dans l'expérience E3. L'économie totale est de 2 185 tokens, soit 42,48 %. 30
5.4 Six observations historiques v12. Chaque barre correspond à une expérience enregistrée ; aucune moyenne interversions n'est utilisée. 32

Liste des tableaux

1.1 Livrables techniques observables du projet. 3
1.2 Phases logiques de réalisation du projet. 3

2.1 Comparaison qualitative de stratégies, sans classement universel. 5
2.2 Objectifs du projet et éléments utilisés pour les apprécier. 5
2.3 Bibliothèques déclarées dans `requirements.txt` ; version Python relevée dans `venv/pyvenv.cfg` et version du serveur Ollama observée le 3 octobre 2026. . . . 6

3.1 Routage v12 conservé en v13 lorsque la compression est appliquée. 12
3.2 Rôles fonctionnels ; leur mode d'exécution dépend de la branche. 13

4.1 Principaux composants de la version étudiée. 17
4.2 Routes principales consultables dans `api.py`. 18
4.3 Lecture fonctionnelle des principales zones de l'application. 19
4.4 États de lecture du résultat et action associée. 26

5.1 Lecture de la source du cas court : le prix le plus bas ne suffit pas à rendre un devis admissible. 29
5.2 Essai v13 court du 1er octobre 2026 ; source locale E3, annexe A. 29
5.3 Mesures v12 conservées pour l'analyse historique ; source E1. 31
5.4 Deux comparaisons v13 terminées et deux arrêts distincts ; sources E2 et E3. . . . 32
5.5 Exemples de garanties testées ; les propriétés sémantiques générales restent hors périmètre. 33

C.1 Champs utiles à la lecture des traces d'expérience. 40

Liste des abréviations

Abréviation	Signification
API	Interface de programmation applicative
CPU	Processeur central
GPU	Processeur graphique
JSON	JavaScript Object Notation
LLM	Large Language Model, modèle de langage de grande taille
PFA	Projet de fin d'année
RAG	Retrieval-Augmented Generation, génération augmentée par recherche
SHA-256	Fonction de hachage utilisée pour contrôler l'identité des données
UI	Interface utilisateur
UML	Unified Modeling Language
USD	Dollar américain, unité monétaire des scénarios

Introduction générale

L'utilisation de plusieurs rôles spécialisés permet de répartir une tâche entre des étapes d'analyse, de contrôle et de décision. Dans une implémentation fondée sur des modèles de langage, chaque appel reçoit un contexte et produit une sortie. La répétition des mêmes données dans plusieurs appels peut devenir une source importante de consommation de tokens, même lorsque les documents de départ restent de taille modérée.

La question étudiée est la suivante : *comment réduire le nombre total de tokens d'un workflow multi-agents sans perdre les données nécessaires à la décision et sans présenter une réponse contradictoire comme validée ?* Le terme de qualité doit ici être défini précisément. Le projet contrôle des faits fournis, des contraintes calculables, le choix final et la conservation de la source. Il ne prétend pas certifier toute interprétation narrative.

Le cas d'étude est la sélection du devis admissible le moins cher pour un envoi soumis à des contraintes de budget, de délai, de capacité et d'assurance. Certains besoins supplémentaires sont déclarés explicitement. Ce domaine a été retenu parce qu'il permet de disposer d'une référence calculable en Python et de construire des cas limites vérifiables.

L'approche associe un compresseur déterministe, un registre de faits protégés, des contrôles structurés et une répartition explicite des tâches entre Python et les appels au modèle. Une interface web locale rend les expériences accessibles et expose les rapports, les vérifications et les coûts mesurés.

Le mémoire suit cinq étapes. Le premier chapitre présente le cadre du stage et la mission. Le deuxième formalise la problématique, les besoins et les choix techniques. Le troisième décrit l'architecture, la compression et les mécanismes de validation. Le quatrième présente la réalisation et le parcours dans l'application web. Enfin, le cinquième analyse le protocole expérimental, les mesures et leurs limites. Les annexes rassemblent les éléments de traçabilité, de reproduction et de lecture des résultats.

Chapitre 1

Cadre du stage et du projet

Ce chapitre situe la mission au sein d'EMSI Recherche & Innovation et définit les livrables du projet. Le cas logistique fournit un support expérimental pour étudier l'efficacité des échanges entre agents.

1.1 Présentation du cadre d'accueil

Le stage est réalisé au sein d'**EMSI Recherche & Innovation**, sous l'encadrement de **M. Hicham Bouchtib**. Il s'inscrit dans la quatrième année de la formation en Ingénierie Informatique et Réseaux de l'École Marocaine des Sciences de l'Ingénieur. Le sujet retenu est *Token-Efficient Cooperative LLM-based Multi-Agent Systems*.

La mission relève de la recherche appliquée : construire une solution exécutable, définir une comparaison mesurable et examiner les conditions dans lesquelles la réduction de contexte reste compatible avec les informations nécessaires à une décision. Le livrable central est donc un démonstrateur instrumenté, accompagné de ses contrôles et de ses traces d'expérimentation. Les données de devis sont fictives et servent à rendre les décisions vérifiables ; elles ne proviennent pas d'une activité commerciale de l'organisme.

1.2 Positionnement du sujet de stage

Le projet relie trois dimensions de l'ingénierie informatique. La première concerne l'orchestration : attribuer des rôles complémentaires aux agents et définir leurs dépendances. La deuxième concerne la représentation de l'information : conserver la source utile sans retransmettre un historique verbeux à chaque étape. La troisième concerne la mesure : vérifier que le gain annoncé inclut tous les appels nécessaires et qu'il accompagne un résultat conforme aux contraintes déclarées.

L'intérêt du stage est de traiter ces dimensions ensemble. Un contexte plus court, une réponse correcte et un parcours utilisateur simple sont trois objectifs distincts ; l'application doit permettre de les observer séparément.

1.3 Mission et livrables

La mission technique consiste à réaliser un environnement d'expérimentation permettant de comparer plusieurs façons de transmettre et de traiter le contexte. Les livrables disponibles sont

le workflow exécutable, le tableau de bord local, les contrôles Python, les tests logiciels et les enregistrements JSON des expériences.

Livrable	Fonction	Élément vérifiable
Workflow	Exécuter les rôles et leurs dépendances	Graphe LangGraph
Compresseur	Réduire les répétitions de contexte	Projection reconstruite depuis la source
Validation	Bloquer les contradictions contrôlables	Tests et rapports rejetés conservés
Dashboard	Lancer et consulter une comparaison	Trois branches et leurs coûts
Évaluation	Conserver la preuve des mesures	Sources, appels et résultats JSON

TABLE 1.1 : Livrables techniques observables du projet.

1.4 Démarche de réalisation

Le développement s'est organisé autour d'itérations : instrumenter les coûts, protéger les faits, contrôler les rapports, réduire les répétitions, remplacer les contrôles calculables par Python, puis rendre les limites explicites dans l'interface. Cette description reflète les versions et résultats présents dans le dépôt ; elle ne constitue pas un relevé de temps du stage.

Phase	Travail	Validation attendue
1	Formaliser le scénario et la référence	Règles calculables sans LLM
2	Mesurer appels et contexte	Compteurs d'entrée et de sortie disponibles
3	Protéger et valider les informations	Contradictions non transmises
4	Compresser et router les tâches	Comparaison à trois branches
5	Évaluer et simplifier l'interface	Traces enregistrées et parcours vérifié

TABLE 1.2 : Phases logiques de réalisation du projet.

1.5 Périmètre et responsabilités

Le système prend en charge des sources structurées et bornées. Il ne collecte pas de tarifs réels, ne réserve pas de transport et n'évalue pas la solvabilité d'un fournisseur. L'utilisateur reste responsable de la fidélité des champs saisis aux documents et de la déclaration de toutes les exigences critiques. Le prototype vérifie ensuite les relations qu'il sait représenter.

1.6 Conclusion

Le cadre retenu est celui d'une recherche appliquée sur le coût et la traçabilité d'un workflow local. Les livrables rendent la démarche observable : un programme exécutable, une interface, des règles de validation et des mesures conservées. Le chapitre suivant traduit cette mission en besoins et en critères d'évaluation.

Chapitre 2

Cadrage et choix techniques

L'objectif est de réduire le coût total d'un traitement coopératif sans laisser circuler des faits contradictoires. Ce chapitre précise les concepts, compare les stratégies possibles et justifie les choix du prototype.

2.1 Problématique

Un contexte plus court n'implique pas automatiquement une économie globale. Un compresseur LLM peut consommer davantage de tokens qu'il n'en économise en aval. De plus, une sortie courte mais factuellement erronée ne constitue pas une amélioration utile. Il faut donc mesurer le coût total, distinguer les sources de différence et rendre visible l'échec de validation.

Un autre problème apparaît lorsque des opérations simples sont confiées à plusieurs agents : comparer un prix à un plafond, des dates ou une capacité est calculable directement. Le projet conserve son idée de compression et l'associe à une exécution déterministe de ces contrôles.

2.2 Notions utiles : modèle, agent et état

Un **modèle** est le moteur qui génère les réponses. Dans les expériences actuelles, le tag utilisé est `llama3`. Un **agent**, dans ce prototype, correspond à un rôle de traitement doté de consignes et d'entrées spécifiques. Plusieurs rôles utilisent donc le même modèle ; six rôles ne signifient pas six modèles différents.

Le **contexte** est le contenu effectivement envoyé dans un appel. L'état LangGraph est plus large : il contient la source, le contexte actif, les faits protégés, les contrôles et l'historique brut. Conserver un élément dans l'état ne signifie pas le retransmettre à tous les appels. Cette distinction rend possible une mémoire d'audit complète et des messages plus courts.

LangGraph fournit une orchestration explicite par états et nœuds, adaptée au mélange d'étapes déterministes et de génération[1]. Dans ce projet, les dépendances sont programmées ; il ne s'agit pas d'agents entièrement libres de choisir leurs tâches, leurs outils ou leurs destinataires.

2.3 Analyse des approches possibles

Approche	Intérêt	Limite pour ce projet
Contexte complet	Référence simple ; toutes les données sont présentes	Répétitions et appels spécialisés coûteux
Résumé génératif	Reformulation compacte de documents	Coût d'inférence et risque d'omission
Recherche de passages	Sélection de fragments pertinents	La pertinence ne prouve pas la couverture des exigences
Projection déterministe	Représentation contrôlable et sans appel de compression LLM	Ne peut pas supprimer arbitrairement les faits utiles
Projection et outils Python	Réduit le contexte et évite des appels de calcul	Nécessite des règles explicites et limite la généralité

Table 2.1 : Comparaison qualitative de stratégies, sans classement universel.

Le protocole actuel retient la dernière approche. La recherche de passages a existé dans une version antérieure, mais elle ne décrit pas le chemin long actuel : celui-ci remplace trois rôles LLM par Python tout en conservant la source complète dans les appels encore actifs.

2.4 Objectifs et critères d'acceptation

L'objectif général est de diminuer les tokens consommés par le workflow tout en conservant les données nécessaires aux contrôles. Il est décliné en critères qui peuvent être observés dans le code, dans une trace d'exécution ou dans l'interface. Cette définition évite de réduire la réussite du projet à un seul pourcentage.

Objectif	Critère observable	Preuve disponible
Réduire la charge	Le coût total de la branche optimisée est comparé à la référence	Compteurs de chaque appel
Conserver le contexte	La source et les conditions fournisseur sont retrouvées dans les prompts attendus	Audit par reconstruction
Écarter les contradictions	Une affirmation incompatible avec les faits n'est pas transmise	Rapport brut et motif de rejet
Rendre la décision vérifiable	Sélection et explication concordent avec la recommandation Python	Liste de contrôles finale
Maîtriser les cas incomplets	Les exigences inconnues ou non représentables bloquent le traitement	Statut de revue explicite
Faciliter la démonstration	Un exemple se lance avec les paramètres automatiques	Parcours du tableau de bord

Table 2.2 : Objectifs du projet et éléments utilisés pour les apprécier.

2.5 Besoins fonctionnels

1. Choisir un scénario prêt à l'emploi ou saisir de un à douze devis.
2. Déclarer les contraintes et les capacités fournisseur nécessaires.
3. Construire une référence Python à partir des faits protégés.
4. Exécuter les trois branches d'une comparaison avec des réglages communs.
5. Contrôler les rapports structurés avant leur transmission.
6. Valider la sélection et l'explication finale, ou exposer un échec.
7. Afficher les coûts, la décomposition des différences et les limites.
8. Sauvegarder les sources et les sorties, y compris les éléments rejetés.

2.6 Besoins non fonctionnels

L'application doit rester utilisable localement, distinguer les résultats mesurés des hypothèses, conserver la traçabilité et prévenir les conversions silencieuses de données inconnues en succès. La simplicité du parcours est également un objectif : les choix techniques sont secondaires par rapport à la sélection du scénario et à la lecture de la décision.

Le critère de coût est le nombre de tokens rapporté par le moteur. Il n'est pas converti automatiquement en prix, en énergie consommée ou en nombre d'heures CPU. Ces grandeurs nécessiteraient leurs propres instruments de mesure.

2.7 Environnement technique

Composant	Rôle dans le dépôt	Version déclarée
Python	Calculs, validation et API	3.14.7 (configuration du venv)
FastAPI	Endpoints et ressources du dashboard	0.141.1
LangGraph	Graphe de dépendances et état partagé	1.2.11
LangChain	Messages et intégration du modèle	1.4.0
langchain-ollama	Adaptateur `ChatOllama`	1.1.0
Ollama	Service local de modèles	0.35.0 (serveur observé)
Llama 3	Génération des rôles encore actifs	Tag enregistré : `llama3`
HTML, CSS, JavaScript	Interface et consultation des rapports	Fichiers locaux

Table 2.3 : Bibliothèques déclarées dans `requirements.txt` ; version Python relevée dans `venv/pyvenv.cfg` et version du serveur Ollama observée le 3 octobre 2026.

Ces relevés décrivent l'environnement présent lors de la revue finale ; ils ne remplacent pas une capture de l'environnement au moment de chaque essai historique. Le modèle local observé le 3 octobre 2026 est `llama3:latest`, 8,0 milliards de paramètres, quantifié en `Q4_0`. Son empreinte est conservée dans le relevé de l'annexe de reproduction. L'adaptateur `ChatOllama` relie les appels LangChain au service local[2]. FastAPI sert les routes de l'application[4].

Phi-3 et Qwen apparaissent dans des essais historiques du dépôt. Ils ne sont pas trois agents du système actuel et ne sont pas nécessaires à son parcours standard. Une comparaison de modèles au protocole v13 exigerait une nouvelle évaluation identique pour chaque modèle ; aucun classement n'est déduit ici.

2.8 Conclusion

Le projet vise une économie mesurable dans un périmètre contrôlé. La solution est un workflow hybride, et son évaluation doit isoler la contribution du compresseur de celle des appels évités. La conception suivante met ces exigences en relation avec les structures réellement utilisées par le code.

Chapitre 3

Conception et modélisation

La conception distingue la source de référence, la mémoire du workflow et le contexte transmis au modèle. Les diagrammes suivants montrent où intervient la compression, comment les rôles coopèrent et à quels endroits une sortie peut être bloquée.

3.1 Architecture générale

L'interface transmet un scénario à l'API. Celle-ci contrôle sa structure et la revue des exigences, puis lance l'orchestration. Les données d'origine restent séparées du texte généré. Le registre protégé alimente les calculs Python et le compresseur ; les appels au modèle n'ont pas pour rôle de réécrire les valeurs de référence. Les résultats et les métriques sont conservés dans des fichiers JSON locaux.

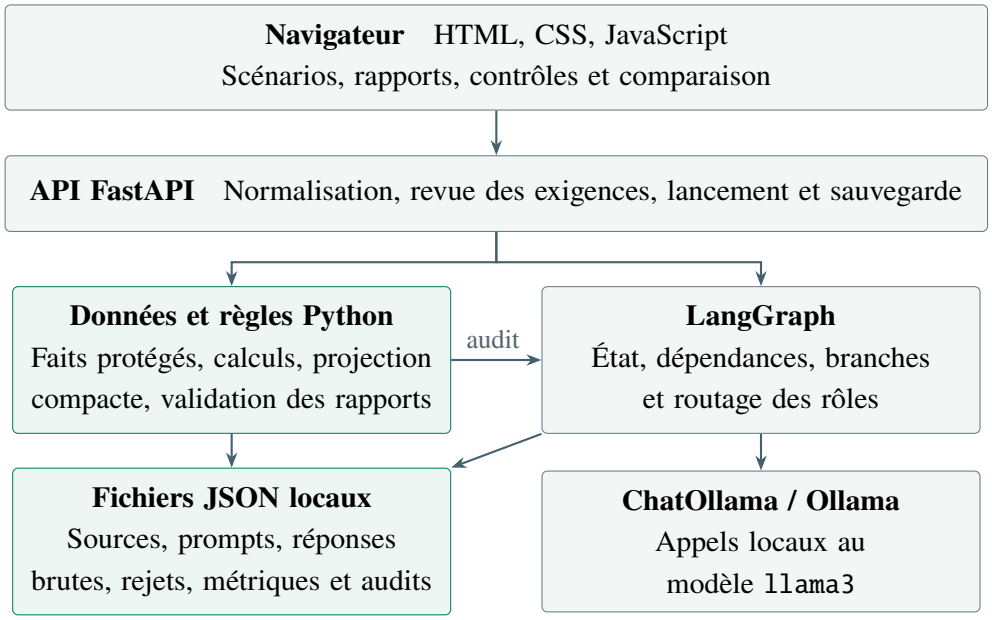


Figure 3.1 : Architecture logique des composants observés dans le dépôt.

L'architecture n'implique ni base vectorielle ni service externe de recherche. L'inférence est locale ; cela n'équivaudrait pas, sans audit complémentaire de l'environnement, à une certification de sécurité de toute la machine. Les fichiers sauvegardés sont des traces de l'application, et non un journal inviolable ou un mécanisme de non-répudiation.

3.2 Cas d'utilisation

L'acteur principal est l'utilisateur du laboratoire d'expérimentation. Il peut choisir un exemple, saisir une source, confirmer ses exigences, lancer une comparaison, consulter les rapports et télécharger les résultats. Les contrôles sont déclenchés par le workflow ; ils ne dépendent pas uniquement d'un message affiché par le navigateur.

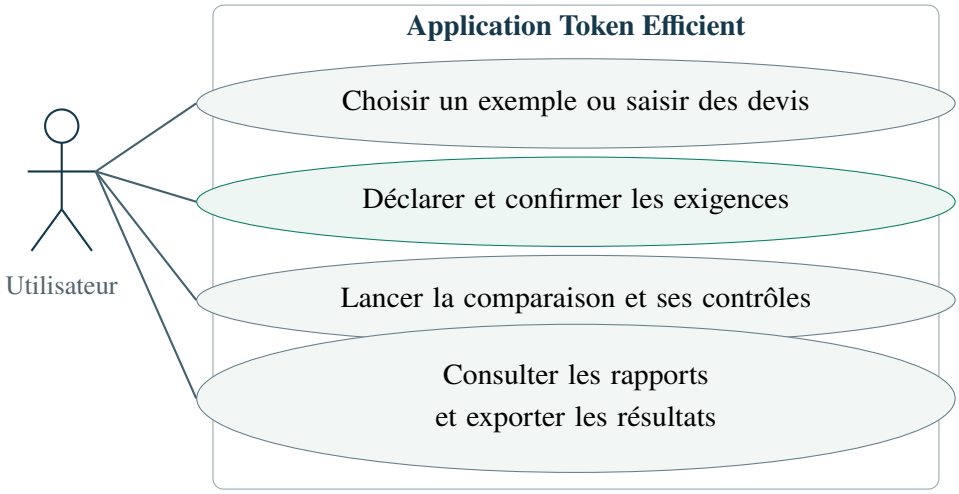


FIGURE 3.2 : Vue simplifiée des cas d'utilisation réellement exposés.

3.3 Modèle des données et état

Le scénario contient une consigne complète, une route, une quantité, un budget, une date limite, l'exigence d'assurance et une collection de devis. Chaque devis porte un identifiant, un fournisseur, un mode de transport, un prix total, une date d'arrivée, une capacité, un indicateur d'assurance et les conditions originales du fournisseur.

Les classes `ProtectedFacts` et `QuoteFacts` sont des dataclasses figées. Les devis sont stockés dans un tuple. La revue supplémentaire est sérialisée de manière canonique ; la source et le registre portent des empreintes SHA-256. Ces empreintes servent à détecter des changements, mais ne constituent pas une signature indépendante attestant la vérité de la source.

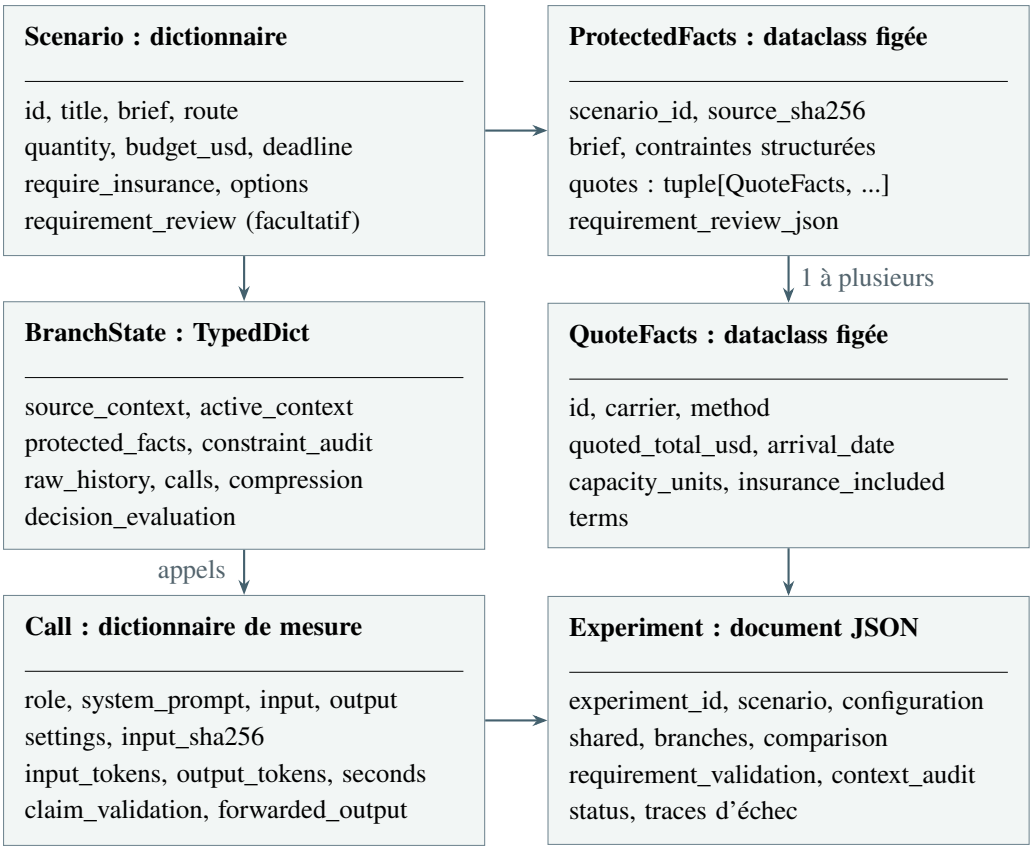


FIGURE 3.3 : Modèle des données : seules ProtectedFacts et QuoteFacts sont des classes métier figées ; les autres blocs sont des structures de données.

L'historique brut utilise un accumulateur dans l'état. Les sorties rejetées y restent consultables. Les prompts en aval utilisent le contexte actif et les versions contrôlées des rapports. Cette séparation évite de confondre la mémoire de l'expérience avec le contenu autorisé à influencer la décision.

3.4 Calcul de référence en Python

Pour le devis i, notons p_i son prix, d_i sa date d'arrivée, q_i sa capacité et s_i son indicateur d'assurance. Les exigences correspondantes sont le budget B, la date limite D, la quantité Q et le booléen S. L'admissibilité de base s'écrit :

$$e_i = (p_i \leq B) \wedge (d_i \leq D) \wedge (q_i \geq Q) \wedge (\neg S \vee s_i). \tag{3.1}$$

Pour un ensemble K de capacités supplémentaires déclarées et prises en charge, la règle devient $e_i^* = e_i \wedge \bigwedge_{k \in K} c_{ik}$. Une valeur inconnue est traitée par la revue préalable ; elle n'est pas assimilée à une capacité présente. Les prix sont comparés avec `Decimal`, et les dates ISO avec les objets de date Python.

L'ensemble des devis admissibles est $E = \{i : e_i^* = \text{vrai}\}$. Si cet ensemble est vide, le résultat est `hold`. Sinon, le calcul détermine les devis de prix minimal :

$$M = \arg\min_{i \in E} p_i. \tag{3.2}$$

En cas d'égalité, la recommandation Python retient l'identifiant lexicographiquement le plus petit. La validation finale exige aussi l'accord avec cette recommandation canonique. Un devis exactement au budget, à la date limite ou à la capacité demandée peut être admissible.

L'objectif n'est donc pas de demander au modèle d'inventer une référence, mais de vérifier sa fidélité à une référence explicite. Cette décision de conception améliore le contrôle factuel et limite la portée des affirmations sur la capacité de raisonnement autonome du modèle.

3.5 Compression déterministe et routage

3.5.1 Ce que le compresseur conserve

Le compresseur actuel est une projection Python. Il conserve la consigne, tous les champs de chaque devis, toutes les conditions fournisseur et, lorsqu'elle existe, la revue des exigences. Les contrôles calculés sont représentés une seule fois dans une table compacte. Cette approche réduit la répétition et la verbosité de la représentation ; elle ne résume pas librement le sens des documents au moyen d'un autre modèle.

Coût de compression

Le protocole actuel n'effectue aucun appel LLM pour compresser. Son coût est nul en tokens d'inférence, mais son exécution Python et sa sérialisation demandent toujours du temps CPU.

En mode automatique, la projection est appliquée si sa longueur en caractères est inférieure à celle de la source accompagnée de son audit développé. Ce test n'est pas une prédiction exacte de tokens. Le gain effectif est calculé après l'inférence à partir des compteurs du moteur. Si la projection n'est pas appliquée, le routage optimisé n'est pas activé et aucune économie de compression n'est revendiquée.

3.5.2 Exemple de représentation compacte

Le mécanisme peut être illustré par le devis S1 du scénario court. Dans la source lisible, les noms de champs sont répétés pour chaque offre. La représentation compacte déclare une fois l'ordre des colonnes, puis sérialise les valeurs. L'extrait suivant est tiré de la source et de la projection enregistrées dans l'expérience E3.

Listing 3.1 : Extrait de la représentation développée du devis S1.

```
QUOTE S1 - Rhone Express / Road courier
All-in total: USD 720
Warehouse arrival: 2026-10-07
Confirmed capacity: 40 units
Transit insurance included: True
```

Listing 3.2 : Extrait de la projection : en-tête commun, valeurs et contrôles compacts.

```
QUOTES: ID | carrier | method | total USD | arrival | capacity | insured
S1|Rhone Express|Road courier|720|2026-10-07|40|true

PYTHON CHECKS (P=PASS, F=FAIL; authoritative):
S1|within_budget=P,on_time=P,enough_capacity=P,insured_as_required=P
```

Les conditions fournisseur et la consigne ne sont pas reproduites dans ces deux extraits pour en faciliter la lecture. Elles sont conservées intégralement dans la projection utilisée par le système. La réduction provient donc d'une représentation moins répétitive et d'un audit plus concis, sans remplacement des conditions par une paraphrase générative.

L'exemple montre aussi la séparation des responsabilités. Le compresseur représente l'information ; le calcul Python établit les valeurs `PASS` ou `FAIL` ; le rôle LLM reçoit ces informations dans son prompt. Le gain réel est mesuré sur ce prompt complet, incluant les consignes du rôle et les rapports autorisés, et non sur le seul extrait affiché ici.

3.5.3 Deux chemins de traitement

Chemin	Critère	Rôles LLM	Rôles Python
Court	Au plus 2 devis et 2 600 caractères de source	Ingestion, Logistique, Risque, Décision	Finance, Conformité
Long	Autres entrées acceptées	Ingestion, Risque, Décision	Logistique, Finance, Conformité

Table 3.1 : Routage v12 conservé en v13 lorsque la compression est appliquée.

Le seuil court/long est une heuristique déclarée et modifiable ; il n'a pas été calibré pour garantir un pourcentage minimal. Les rôles remplacés restent des étapes du graphe, mais ils exécutent une fonction Python au lieu d'un appel au modèle. Sur le chemin long, les contrôles déjà présents dans la table ne sont pas répétés dans des rapports intermédiaires supplémentaires transmis aux agents. Les rapports Python séparés restent sauvegardés.

3.6 Orchestration LangGraph

L'ingestion est exécutée en amont des branches et partagée. Dans chaque branche, Logistique et Finance précèdent le couple Risque–Conformité ; Décision attend les deux derniers rapports. Les étapes d'une même paire sont indépendantes du point de vue du graphe. Le débit réel des appels locaux dépend néanmoins de la capacité et de la configuration du serveur Ollama.

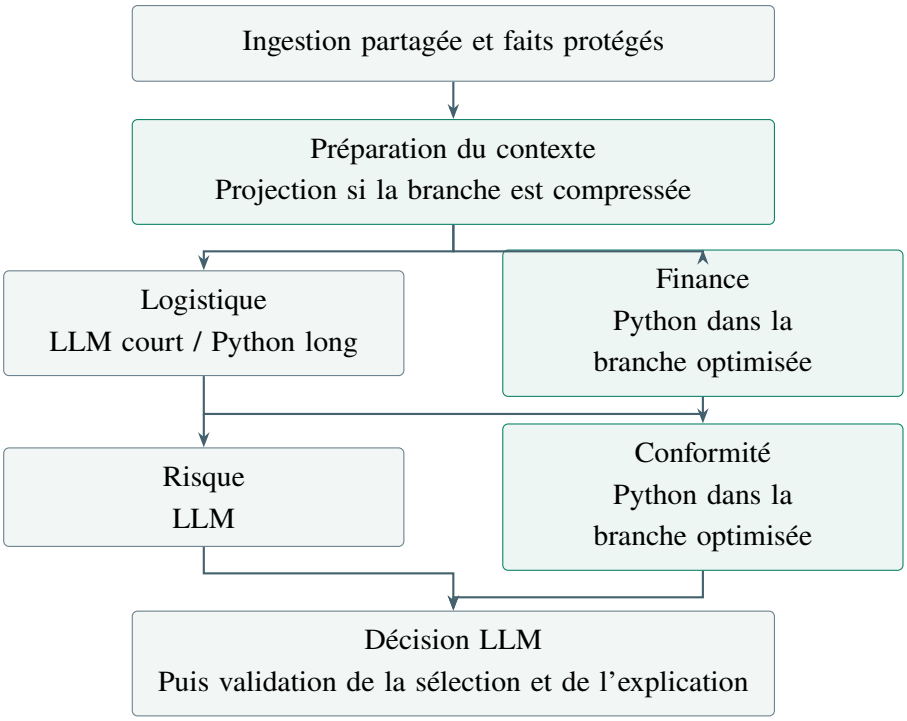


FIGURE 3.4 : Dépendances du graphe et exécution hybride. Dans les deux branches de contrôle, les six rôles restent des appels LLM.

Les agents ne reçoivent pas automatiquement tout l'historique de l'état. Le code construit leurs messages à partir du contexte actif et des rapports validés. Cette construction explicite est une condition nécessaire pour attribuer la réduction à une modification identifiable de la charge transmise.

3.6.1 Responsabilités des rôles

Les noms des agents désignent des responsabilités dans le graphe. Ils ne correspondent pas à des modèles distincts. Le tableau 3.2 précise leur contribution et permet de lire les rapports proposés par l'interface.

Rôle	Responsabilité	Sortie ou contrôle principal
Ingestion	Première lecture partagée des faits fournis	Rapport structuré contrôlé
Logistique	Délai et capacité du transport	Contraintes `on_time` et `enough_capacity`
Finance	Comparaison du prix au budget	Contrainte `within_budget`
Risque	Examen des éléments autorisés et des réserves	Affirmations et incertitudes contrôlées
Conformité	Vérification de l'assurance exigée	Contrainte `insured_as_required`
Décision	Restitution de la recommandation de référence	Sélection, montant, date et justification

TABLE 3.2 : Rôles fonctionnels ; leur mode d'exécution dépend de la branche.

Les fonctions Python de Finance, Logistique et Conformité ne tentent pas d'interpréter librement les documents. Elles exécutent les règles déclarées. Inversement, une sortie LLM n'acquiert pas l'autorité d'un fait simplement parce qu'elle est formulée par un agent spécialisé : elle reste soumise au contrôle.

3.7 Barrières de validation

3.7.1 Revue des exigences avant inférence

Les exemples intégrés, inchangés, possèdent un périmètre connu de quatre règles. Pour une source personnalisée, l'utilisateur déclare les besoins et confirme leur complétude. Les capacités supplémentaires prises en charge sont le transport réfrigéré, la manutention fragile et la signature à la livraison. Pour chaque devis et capacité requise, la valeur doit être connue.

Une confirmation absente, une capacité requise inconnue ou une condition supplémentaire non prise en charge entraîne `needs_review`. Aucune décision LLM n'est alors produite et aucun pourcentage d'économie n'est calculé. En particulier, déclarer du transport réfrigéré ne démontre pas le respect d'une plage précise de 2 à 8 °C.

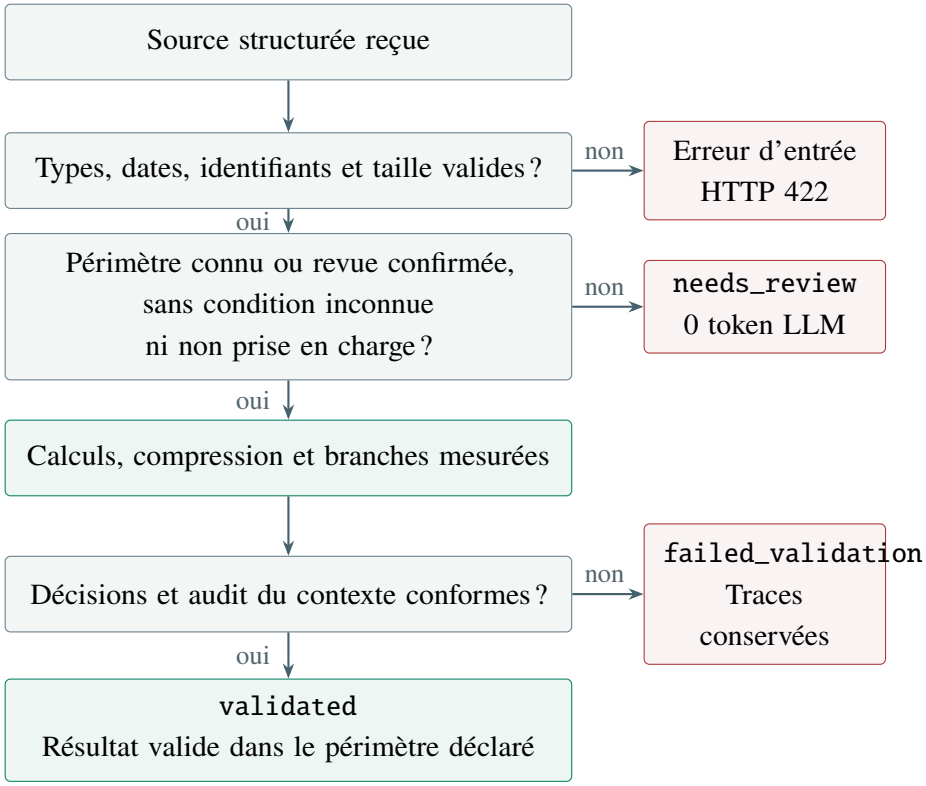


FIGURE 3.5 : Contrôle des entrées et du résultat ; les erreurs d'exécution constituent un état d'échec distinct.

La confirmation est une déclaration humaine. L'application ne peut pas prouver qu'une exigence présente uniquement dans la prose a été correctement reportée dans le formulaire. Cette responsabilité n'est pas supprimée par l'existence d'une case de confirmation.

3.7.2 Contrôle des rapports et des contradictions

Les sorties intermédiaires suivent un schéma JSON avec `claims` et `uncertainties`. Une affirmation comporte un identifiant de devis, une contrainte, un résultat et une courte explication. L'accord du booléen seul ne suffit pas : l'explication doit aussi respecter la forme canonique attendue. Les incertitudes transférées doivent correspondre aux réserves autorisées issues de la source.

Par exemple, l'énoncé « 12 800 dépasse 15 000 » contredit une comparaison Python. Il ne doit pas influencer un agent suivant. Le rapport original et les motifs du rejet restent enregistrés, tandis que `forwarded_output` contient le texte autorisé. Cette politique conservatrice peut également rejeter une reformulation raisonnable qui ne correspond pas à la forme admise.

3.7.3 Validation de la décision

Le prompt final fournit les champs de recommandation et une explication canonique calculée par Python. Le modèle doit les reproduire sans ajouter de conditions non vérifiées. Une mauvaise sélection ou une explication modifiée est marquée comme échec ; le système ne corrige pas silencieusement cette sortie pour la présenter comme une décision réussie.

L'audit de contexte reconstruit le registre, la projection, les rapports déterministes et les prompts attendus. Il vérifie que les sources nécessaires et les déclarations supplémentaires figurent dans les appels actifs. Cette vérification établit une propriété de conservation et de cohérence, et non une preuve que le modèle a compris toute la signification du texte.

3.8 Séquence d'une comparaison

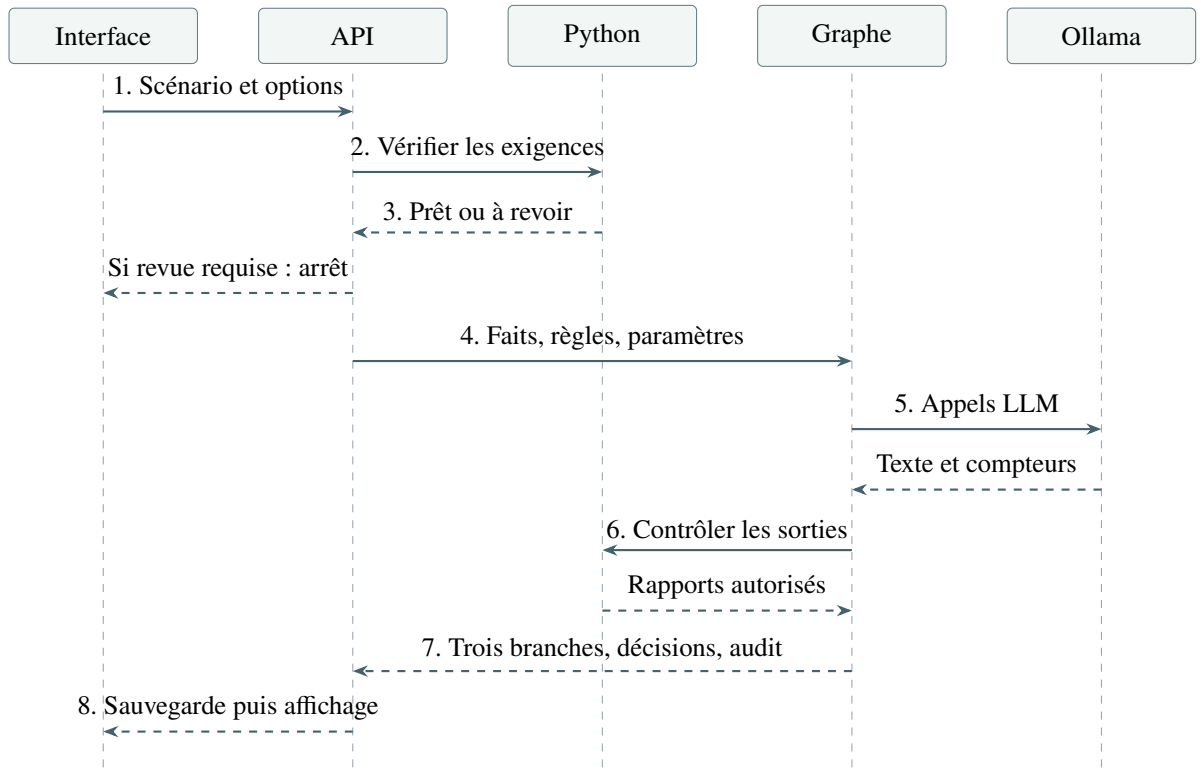


FIGURE 3.6 : Séquence simplifiée. Les étapes 5 et 6 se répètent selon les rôles ; les branches partagent l'ingestion.

3.9 Conclusion

La conception distingue les faits de référence, le contexte actif et les sorties générées. Le compresseur reste central pour supprimer des répétitions, tandis que Python évite de déléguer des contrôles déterministes au modèle. Les diagrammes représentent cette implémentation hybride et non une chaîne dans laquelle différents modèles échangeraient librement des messages.

Chapitre 4

Réalisation et présentation de l'application

Ce chapitre relie les composants logiciels au produit visible. Il présente l'organisation du backend et le parcours permettant de préparer une source, consulter les rapports et examiner une comparaison enregistrée.

4.1 Organisation du code

La réalisation sépare la présentation web, les règles de domaine, l'orchestration et les mesures. Le tableau suivant donne des points d'entrée pour relier les descriptions de ce rapport à l'implémentation.

Fichier	Responsabilité
`api.py`	Routes FastAPI, normalisation et sauvegarde
`index.html`, `script.js`	Interface et consultation des expériences
`custom_scenario.js`	Formulaire personnalisé et confirmation
`protected_facts.py`	Registre immuable et empreintes
`quote_calculations.py`	Calculs de référence
`requirement_review.py`	Capacités, confirmation et arrêt pour revue
`lossless_projection.py`	Projection compacte de la source
`tool_routing.py`	Choix court/long et rôles Python
`langgraph_orchestration.py`	État et graphe d'une branche
`graph_verified_lossless_workflow.py`	Exécution, compteurs et comparaison
`claim_guard.py`	Filtrage des rapports transmis
`context_integrity_audit.py`	Reconstruction et contrôle indépendant des traces

Table 4.1 : Principaux composants de la version étudiée.

Les variantes anciennes sont conservées dans le dépôt pour comprendre l'évolution du prototype. Les choix *Legacy* ont été retirés du dashboard. Leur présence dans les fichiers ne signifie pas qu'ils participent aux comparaisons actuelles.

4.2 API et interface

Route	Utilisation
`GET /api/scenarios`	Liste disponible ; l'interface ne présente que les scénarios actuels
`POST /api/compare`	Exécute ou bloque une comparaison, puis enregistre la trace
`GET /api/compare/template/{id}`	Exemple éditable pour une source personnalisée
`GET /api/compare/latest/{id}`	Dernière comparaison sauvegardée du protocole demandé
`GET /api/evaluation/latest`	Synthèse d'évaluation enregistrée pour le protocole

TABLE 4.2 : Routes principales consultables dans `api.py`.

Le parcours standard consiste à choisir un exemple court, moyen ou détaillé, puis à cliquer sur *Run comparison*. L'interface utilise par défaut le mode *Automatic*, associé au protocole Adaptive v13. Les autres stratégies et la sélection du rapport affiché restent dans *Advanced settings*. L'API conserve néanmoins `complete` comme valeur par défaut pour un client qui omet le paramètre `fidelity` ; un client souhaitant v13 doit transmettre explicitement `adaptive`.

La source personnalisée regroupe l'envoi, les devis et une confirmation finale. Les capacités fournisseur ne sont affichées que lorsqu'elles sont requises. Une modification des données efface la confirmation, et un champ obligatoire masqué dans une section repliée est révélé lors de la validation du formulaire. La vérification côté serveur reste nécessaire : masquer un contrôle d'interface ne doit pas permettre de contourner une règle.

4.3 Présentation du tableau de bord

4.3.1 Une interface organisée autour de l'expérience

Le site web constitue le point d'entrée du démonstrateur. Sa navigation latérale distingue l'espace d'expérimentation et la comparaison des branches. L'en-tête situe l'objectif de compression, tandis que les trois indicateurs supérieurs affichent les tokens, la durée et la réduction observée. La partie centrale associe un panneau de préparation à gauche et un panneau de rapports à droite.

Cette organisation rapproche l'action et son résultat. L'utilisateur peut choisir un exemple, lancer son traitement, puis lire les rapports sans passer par plusieurs pages de configuration. Les paramètres détaillés restent accessibles dans *Advanced settings*, tandis que le mode *Automatic* est sélectionné par défaut.

4.3.2 Choisir un exemple et préparer une comparaison

Trois exemples intégrés représentent des tailles différentes : deux devis, quatre devis et six devis détaillés. L'utilisateur peut ouvrir *View scenario details* pour lire les données avant le lancement.

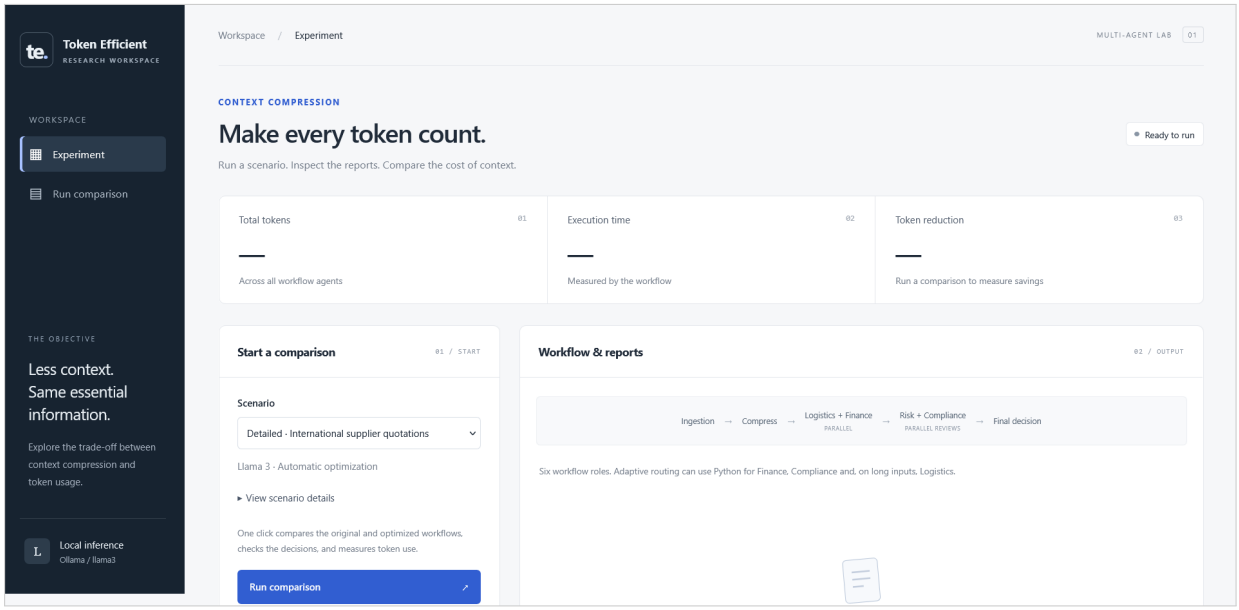


FIGURE 4.1 : Vue générale du tableau de bord. Le panneau de préparation, les indicateurs et l'espace des rapports forment le parcours principal.

Zone de l'écran	Action ou information	Intérêt pour la démonstration
Navigation latérale	Accéder à l'expérience ou à sa comparaison	Conserver des repères de lecture
Indicateurs supérieurs	Lire coût, durée et réduction	Présenter les mesures essentielles
Start a comparison	Choisir la source et lancer l'exécution	Limiter la préparation du cas standard
Workflow & reports	Suivre les rôles et sélectionner un rapport	Examiner le contenu produit
Compare your runs	Comparer les branches et leurs contrôles	Distinguer économie et conformité
Model evaluation	Consulter une synthèse enregistrée	Situer une exécution dans son lot d'essais

TABLE 4.3 : Lecture fonctionnelle des principales zones de l'application.

La commande *Run comparison* exécute les branches du protocole sélectionné ; elle ne se limite pas à appliquer une réduction graphique à un ancien coût.

Pour une démonstration reproductible, le bouton *Load last saved comparison*, placé dans les réglages avancés, permet de relire une expérience existante. Le statut *Saved comparison* distingue cette consultation d'une nouvelle exécution. Ce parcours est utile pour présenter une trace déjà analysée sans attendre de nouveaux appels au modèle.

4.4 Consulter la décision et les rapports des agents

Les captures de résultats de ce chapitre proviennent d'une nouvelle exécution du scénario court, réalisée le 3 octobre 2026 et conservée sous la référence E5 (annexe A). Les trois totaux en tokens et la sélection coïncident avec ceux de l'expérience E3 du 1er octobre, étudiée au chapitre 5. Les durées affichées correspondent cependant à E5 : cette illustration de l'interface ne remplace pas les mesures historiques du chapitre d'évaluation.

Le schéma du workflow affiche l'enchaînement des rôles. Après traitement ou chargement d'une expérience, les onglets donnent accès aux rapports. Le rôle Décision présente une sélection structurée ; les autres rapports permettent d'examiner les affirmations intermédiaires ou les calculs Python associés. La fonction *Copy report* facilite la reprise d'un résultat précis.

Dans le cas court étudié, la décision attendue porte sur S1 : 720 USD et une arrivée le 7 octobre 2026. La sélection s'explique par le respect du délai, alors que S2 est moins cher mais arrive trop tard. Cette lecture du rapport doit être rapprochée du tableau de décision, plutôt que déduite du seul pourcentage de réduction.

L'interface ne transforme pas toutes les sorties du modèle en résultats validés. Un rapport contradictoire est conservé dans la trace de recherche, mais seule sa version autorisée peut être utilisée en aval. Pour la décision finale, l'échec des vérifications est visible comme un échec ; une sélection incorrecte ne doit pas être présentée comme réussie après correction silencieuse.

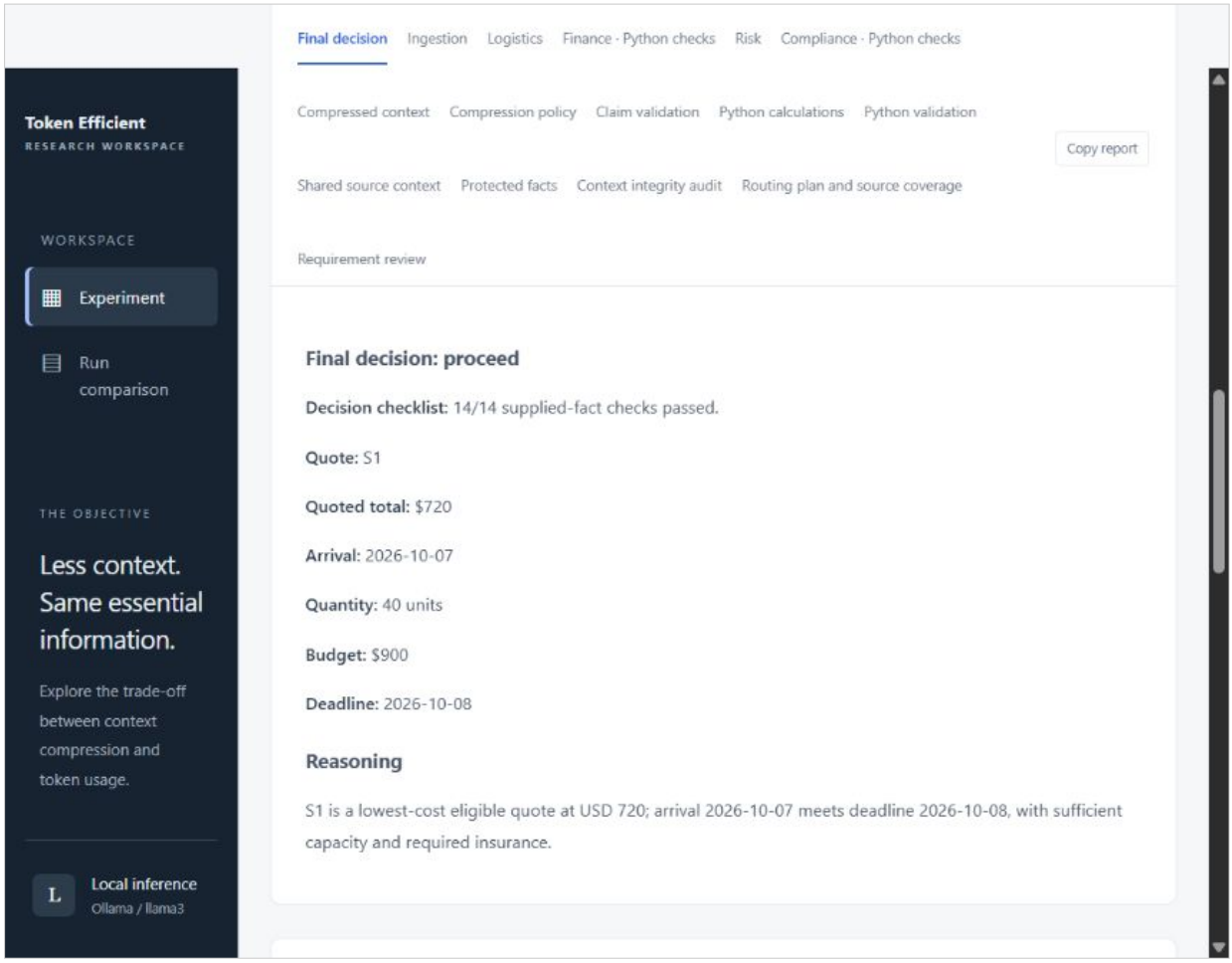


FIGURE 4.2 : Lecture de la décision dans l'application. La sélection, le coût et la date doivent être interprétés avec les contrôles de l'expérience correspondante.

4.5 Comparer les coûts et examiner les contrôles

4.5.1 Les trois branches visibles

La zone *Compare your runs* rassemble les coûts, les durées, les contrôles et le statut de chaque branche. En mode adaptatif, la présence d'une branche *Compression only* est essentielle : elle permet d'examiner ce qui change lorsque le contexte est compacté tout en conservant les six rôles LLM.

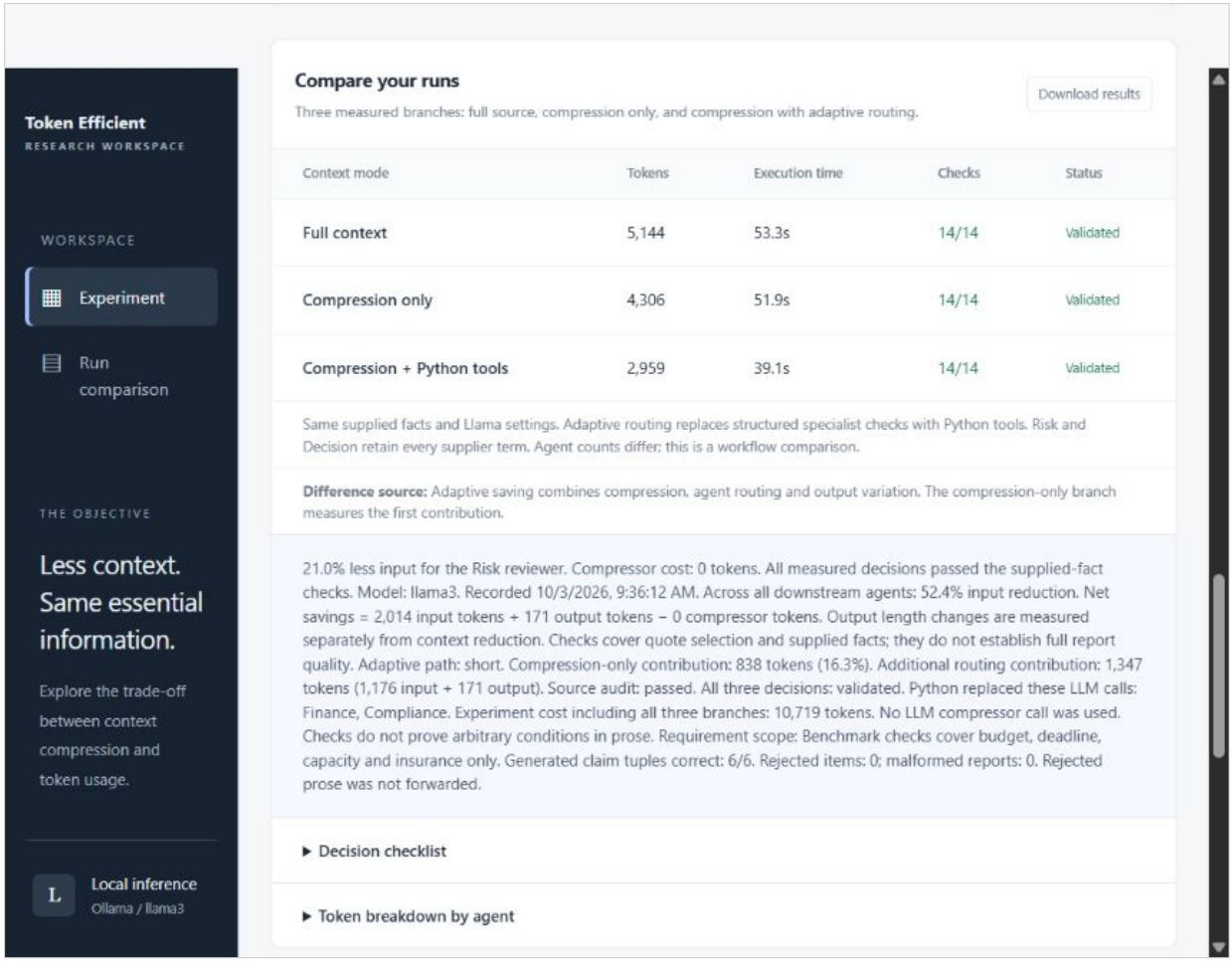


Figure 4.3 : Comparaison des trois branches. Les totaux mesurés sont accompagnés de leur statut et de l'origine des différences.

Le taux affiché pour la branche optimisée mesure une réduction totale. Il associe l'effet du contexte compact, celui des appels remplacés par Python et les variations de longueur des réponses. Les valeurs détaillées et les formules du chapitre 5 rendent cette attribution explicite. Un taux élevé ne remplace donc pas l'examen des contrôles.

4.5.2 Liste de vérification de la décision

La section dépliable *Decision checklist* expose les contrôles comparés. Elle permet au lecteur de vérifier la sélection et les faits attendus, au lieu de s'appuyer uniquement sur un statut de synthèse. Pour l'expérience courte E3, les trois branches satisfont les quatorze contrôles enregistrés.

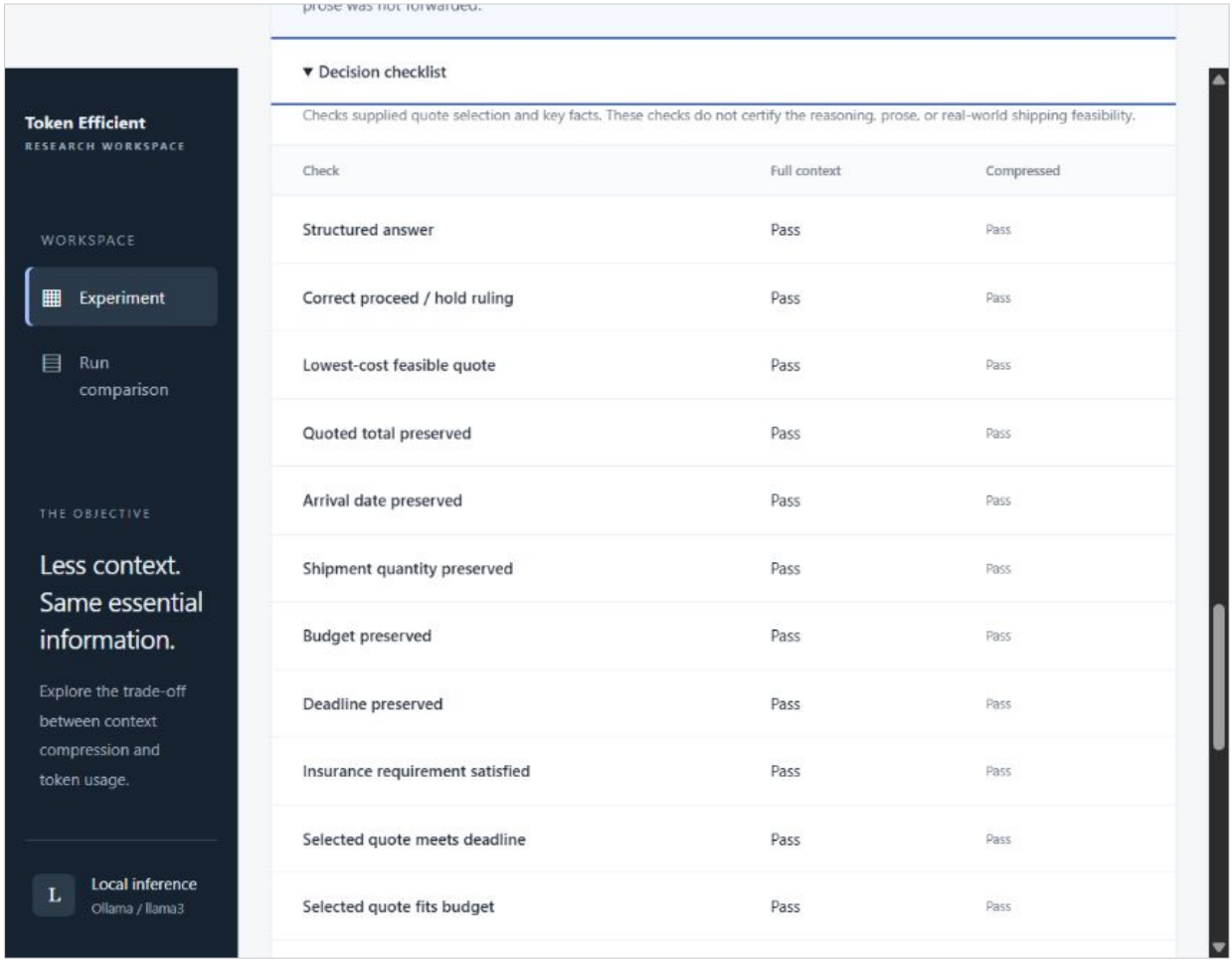


FIGURE 4.4 : Extrait des contrôles détaillés de la décision. La liste rend les vérifications examinables ; la capture montre une partie des quatorze contrôles. Ceux-ci ne certifient pas la compréhension de toute formulation libre.

4.5.3 Tokens par rôle et export de la trace

La section *Token breakdown by agent* distingue les entrées et les sorties des rôles. Elle aide à comprendre pourquoi la suppression d'un appel spécialisé ne doit pas être attribuée entièrement au compresseur. Un rôle Python peut produire un rapport utile tout en consommant zéro token d'inférence.

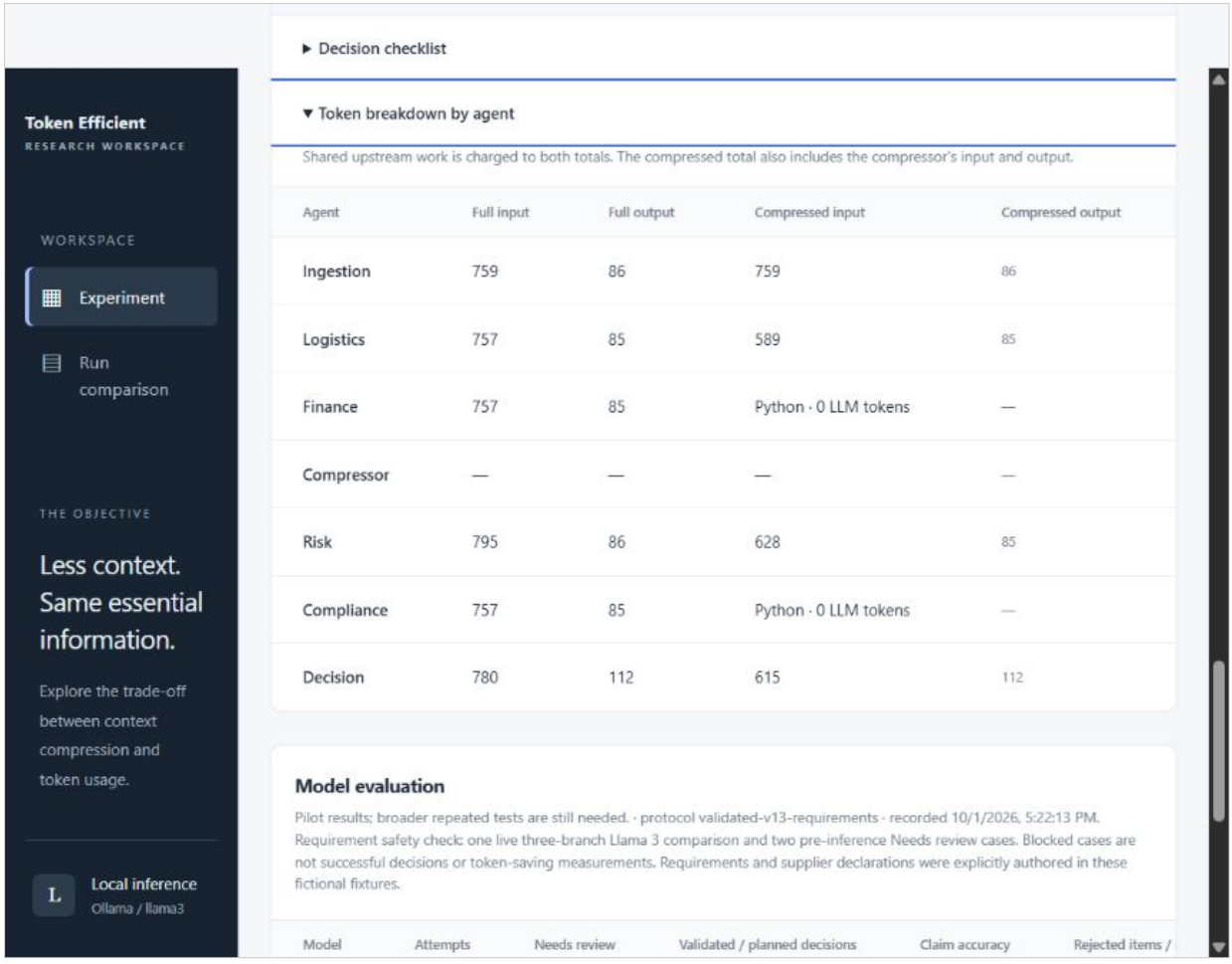


Figure 4.5 : Répartition des tokens par rôle. La décomposition des entrées et sorties complète le total affiché en tête du dashboard.

Le bouton *Download results* exporte l'expérience JSON. Les mesures présentées dans le mémoire sont reliées à ces enregistrements, et non seulement à une capture d'écran. L'annexe C décrit les champs utiles à l'examen de cette preuve.

4.6 Construire un scénario personnalisé

4.6.1 Saisie de l'envoi et des offres

Le formulaire personnalisé est organisé en trois étapes : l'envoi, les devis fournisseur, puis la confirmation. La première conserve la demande complète, la route, la quantité, le budget, la date

limite et l'assurance. La deuxième décrit chaque offre, avec ses conditions originales. Un exemple éditable peut être chargé pour faciliter la saisie.

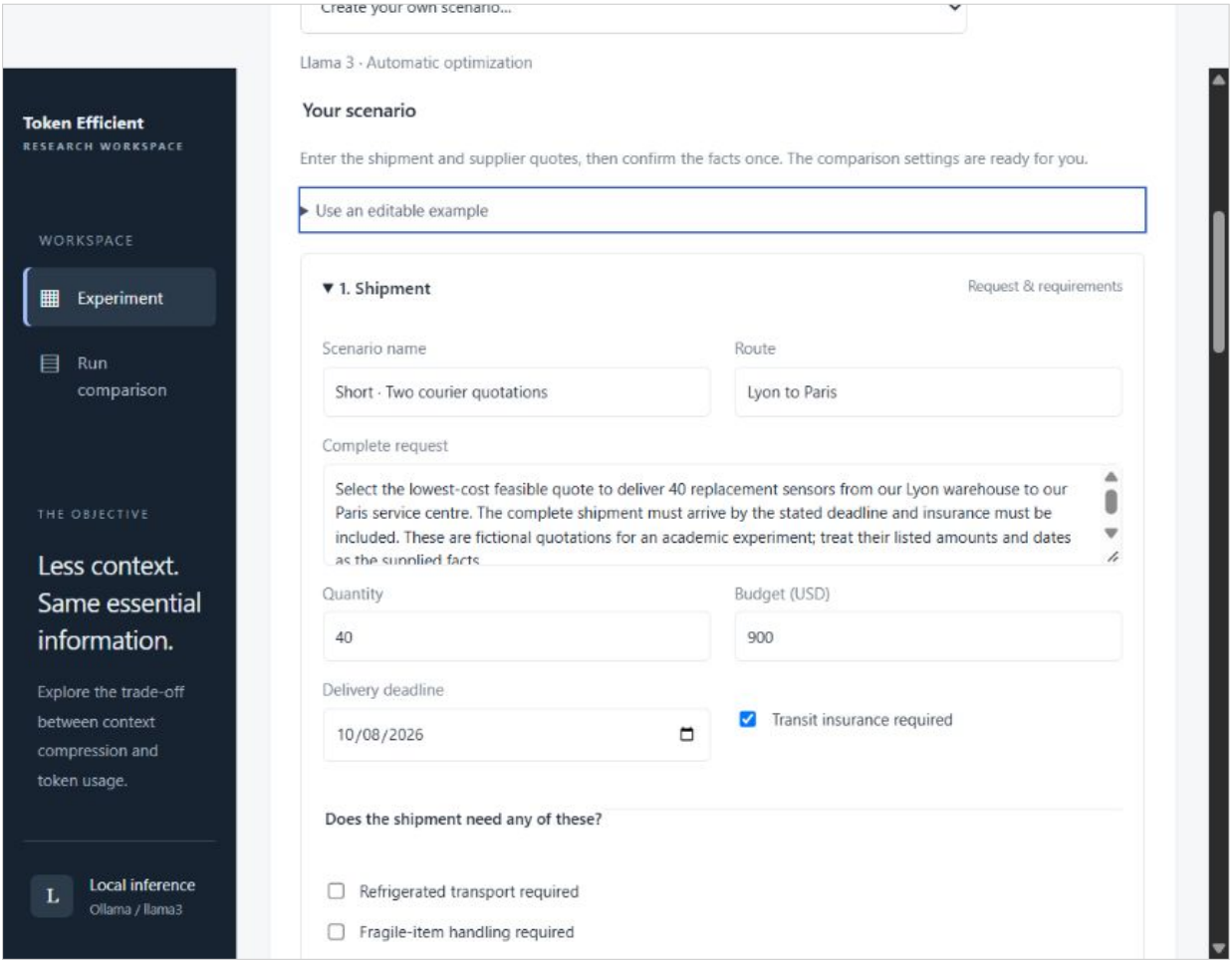


FIGURE 4.6 : Préparation d'un scénario personnalisé. Extrait de la première étape après chargement de l'exemple éditable : demande, route, quantité, budget, délai et assurance. La saisie des offres et la confirmation se poursuivent plus bas dans le formulaire.

La présence d'un texte libre ne suffit pas à rendre toute exigence calculable. Les besoins supplémentaires pris en charge doivent être déclarés dans les champs prévus. Lorsqu'une capacité est requise, les contrôles correspondants deviennent visibles pour chaque fournisseur ; une valeur inconnue reste distincte d'une réponse négative ou positive.

4.6.2 Revue des exigences et responsabilité de la saisie

La confirmation finale indique que l'utilisateur a vérifié la complétude des contraintes et leur correspondance avec la source. Toute modification efface cette confirmation. Cette règle évite d'appliquer une ancienne validation humaine à des données qui ont changé.

Une exigence non prise en charge ou une capacité essentielle inconnue conduit à une revue avant l'inférence. Le formulaire et le message de résultat orientent l'utilisateur vers les informations à compléter. Le système ne remplace pas une absence de donnée par une hypothèse favorable.

4.7 Comprendre les statuts affichés

La présentation distingue les situations où une mesure existe de celles où aucune comparaison ne peut être interprétée. Cette séparation est nécessaire pour éviter qu'un arrêt précoce soit présenté comme une économie maximale.

Situation	Ce que l'utilisateur doit comprendre	Suite du parcours
Prêt à exécuter	Aucun résultat de cette sélection n'est encore affiché	Choisir puis lancer ou charger
Comparaison sauvegardée	Une trace existante est consultée	Lire son contexte et ses mesures
Validation réussie	Les contrôles déclarés sont satisfaits	Examiner la décision et exporter
Revue nécessaire	Aucune décision automatique n'a été émise	Compléter ou revoir les exigences
Échec de validation	Une sortie contredit les contrôles attendus	Examiner la trace d'échec
Compression ignorée	Aucune économie n'est attribuée au compresseur	Lire le contexte conservé

Table 4.4 : États de lecture du résultat et action associée.

4.8 Apport de l'interface à la démarche de recherche

Le site ne se limite pas à afficher une réponse finale. Il rapproche une source, des rapports, des compteurs et des contrôles. Le lecteur peut ainsi passer du résultat synthétique à une preuve plus détaillée, puis exporter la trace pour une analyse indépendante.

Le parcours standard réduit le nombre de choix techniques nécessaires, tandis que les réglages avancés conservent la possibilité d'étudier les branches. Cette organisation répond à deux usages : une démonstration compréhensible pour le jury et une expérimentation plus détaillée pour l'auteur du projet.

4.9 Conclusion

L'application web expose le workflow sous la forme d'un parcours contrôlé : choisir les données, exécuter ou charger une comparaison, examiner la décision, puis consulter les preuves. La séparation entre interface, calculs et orchestration permet de faire évoluer la présentation sans déplacer l'autorité des règles vers le navigateur. Le chapitre suivant analyse les résultats mesurés de cette réalisation.

Chapitre 5

Évaluation expérimentale et discussion

L'évaluation répond à deux questions : quelle économie a été mesurée et quelles propriétés du résultat ont été vérifiées ? Les coûts des trois branches sont comparés sur la même source ; les succès, les arrêts pour revue et les limites du protocole restent distingués.

5.1 Protocole de mesure

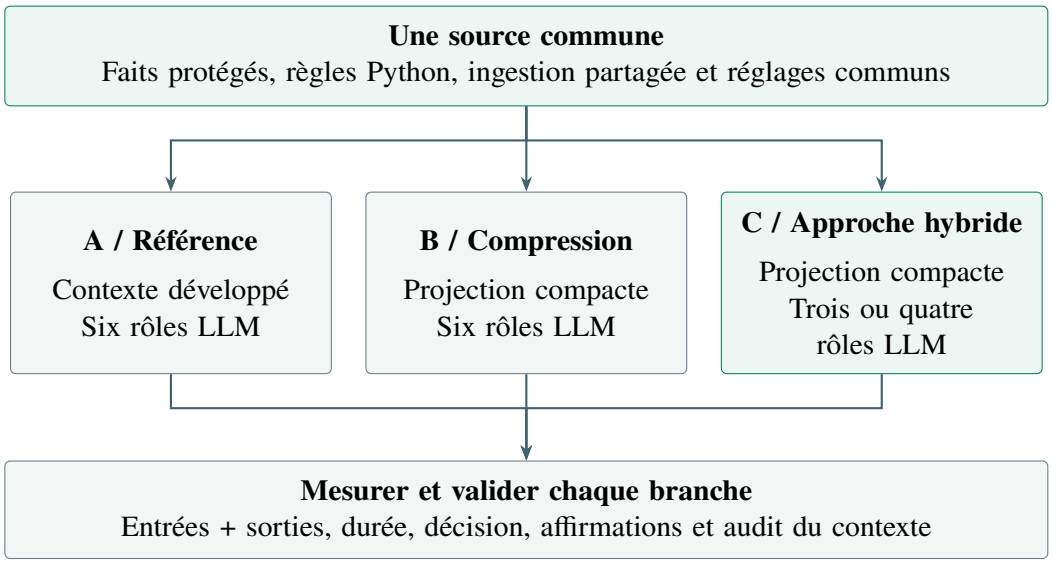


Figure 5.1 : Plan de comparaison : la branche B sépare la compression de l'effet additionnel des outils Python.

5.1.1 Trois branches et une ingestion partagée

Les trois branches partagent la source et le résultat de l'ingestion. Chaque coût de branche inclut cette ingestion, comme si cette branche était déployée seule. Physiquement, l'ingestion n'est appelée qu'une fois dans l'expérience. La comparaison complète coûte donc davantage qu'un seul déploiement optimisé.

Les réglages enregistrés dans les essais présentés sont le modèle `llama3`, une température de 0, une fenêtre de 16 384 tokens et une limite de sortie de 256 tokens pour Ingestion et de 384 pour chacun des autres rôles LLM. La limite déclarée de 192 pour Compressor n'est pas consommée par la projection déterministe, qui ne l'appelle pas. Les graines des répétitions historiques sont 120

et 121 ; les contrôles v13 utilisent leurs propres graines enregistrées. Une température nulle ne supprime pas la nécessité de répétitions.

La vérification préalable de taille est conservatrice : elle utilise les octets UTF-8 et une marge de sécurité avant l'appel, puis contrôle les compteurs mesurés. Ce n'est pas un tokeniseur exact du modèle. Les entrées trop grandes doivent être rejetées ou réduites explicitement, sans être tronquées en silence.

5.1.2 Coûts et pourcentages

Pour un appel j, le coût mesuré est la somme des tokens d'entrée et de sortie :

$$T_j = T_{j,\text{entre}} + T_{j,\text{sortie}}. \tag{5.1}$$

Le code utilise les champs `prompt_eval_count` et `eval_count` retournés par Ollama, documentés par son API[3]. Une absence de compteurs valides ne doit pas être remplacée par une estimation présentée comme une mesure.

Notons T_B le coût du contexte complet, T_C celui de la compression seule et T_A celui de la compression avec outils. La réduction totale est :

$$R_A = 100\frac{T_B - T_A}{T_B}. \tag{5.2}$$

La décomposition arithmétique est exacte :

$$T_B - T_A = \underbrace{T_B - T_C}_{\text{comparaison compression seule}} + \underbrace{T_C - T_A}_{\text{comparaison avec routage}}. \tag{5.3}$$

Chaque différence peut inclure une variation de longueur des sorties. Elle ne doit donc pas être confondue avec une mesure pure de réduction des entrées. Le fichier d'expérience expose séparément les tokens d'entrée et de sortie économisés.

Si T_I est le coût de l'ingestion partagée et qu'aucun appel additionnel n'est exclu du bilan, le coût physique des trois branches est :

$$T_{\text{exprience}} = T_B + T_C + T_A - 2T_I. \tag{5.4}$$

En pratique, le bilan physique est établi à partir des appels réellement enregistrés, y compris leurs coûts en cas d'échec. Les temps affichés pour les branches et la durée totale de l'expérience sont également distincts ; des appels indépendants peuvent se chevaucher.

5.1.3 Qualité et couverture

Trois dimensions sont séparées : la proportion de décisions ayant satisfait les contrôles, l'exactitude des affirmations structurées évaluées, et le succès de l'audit de contexte. Une valeur parfaite sur un petit échantillon ne constitue pas une garantie statistique pour de nouvelles sources.

Les arrêts `needs_review` sont comptés séparément. Ils n'ont ni décision émise ni économie mesurée. Dans la synthèse du dashboard, le dénominateur des décisions planifiées inclut les branches non exécutées de ces tentatives. Ce ratio mesure donc aussi la couverture automatique. Il ne doit pas être nommé « exactitude des décisions émises » sans cette précision.

5.2 Cas d'étude court : décision et tokens

Le scénario court demande le transport de 40 capteurs de Lyon à Paris, avec un budget de 900 USD, une arrivée au plus tard le 8 octobre 2026 et une assurance obligatoire. S1 coûte 720 USD et arrive le 7 octobre. S2 coûte 540 USD mais arrive le 10 octobre. La capacité et l'assurance conviennent dans les deux offres. S1 est donc le devis admissible le moins cher, même si S2 est moins cher dans l'absolu.

Devis	Prix (USD)	Capacité	Arrivée	Admissibilité
S1 / Rhone Express	720	40	7 octobre	Conforme
S2 / Metro Freight	540	40	10 octobre	Délai dépassé

Table 5.1 : Lecture de la source du cas court : le prix le plus bas ne suffit pas à rendre un devis admissible.

Branche	Tokens	Durée de branche	Contrôles
Contexte complet	5 144	60,9 s	14/14
Compression seule	4 306	59,8 s	14/14
Compression et outils	2 959	44,3 s	14/14

Table 5.2 : Essai v13 court du 1er octobre 2026 ; source locale E3, annexe A.

La différence totale est de 2 185 tokens, soit 42,48 %. La compression seule explique 838 tokens, soit 16,29 points de pourcentage relativement à la référence. La différence additionnelle vaut 1 347 tokens. Le bilan global se décompose en 2 014 tokens d'entrée et 171 tokens de sortie économisés. Les trois décisions et l'audit de contexte ont satisfait leurs contrôles.

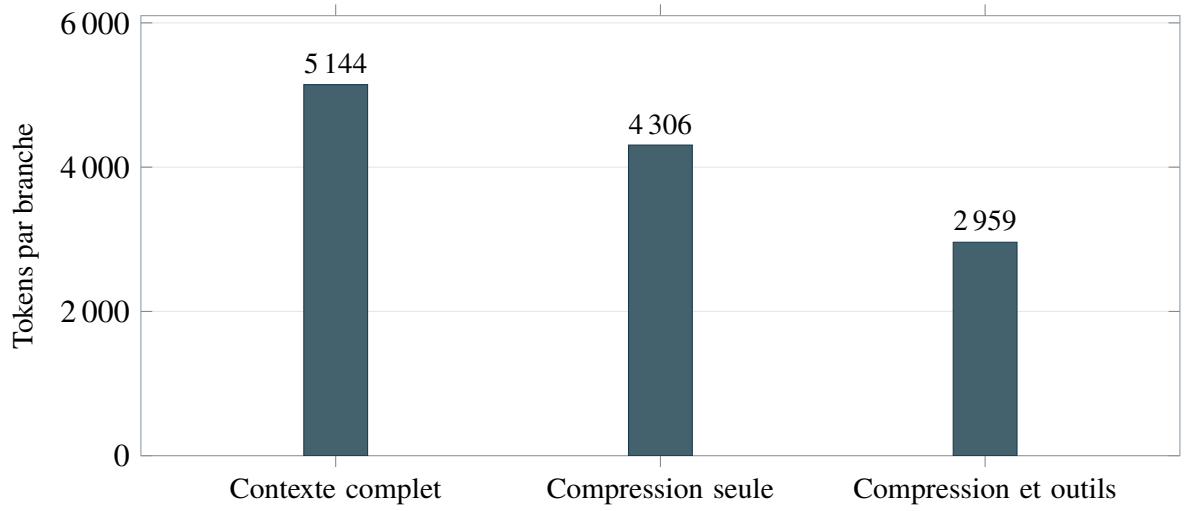


Figure 5.2 : Tokens du cas court v13 : la troisième barre mesure l'optimisation combinée, pas le compresseur seul.

Le coût physique de cette expérience est de 10 719 tokens et sa durée totale de 125,9 secondes. Ces chiffres sont plus élevés que le coût et la durée de la seule branche optimisée, car la démonstration exécute aussi les références.

5.2.1 D'où proviennent les 2 185 tokens économisés ?

La figure 5.3 décompose le total de référence. La partie conservée correspond au coût de la branche optimisée. Les deux autres segments correspondent au gain entre contexte complet et compression seule, puis au gain additionnel entre compression seule et compression avec outils Python. Cette présentation distingue l'effet observé de la compression de celui des appels évités et de la variation des sorties.

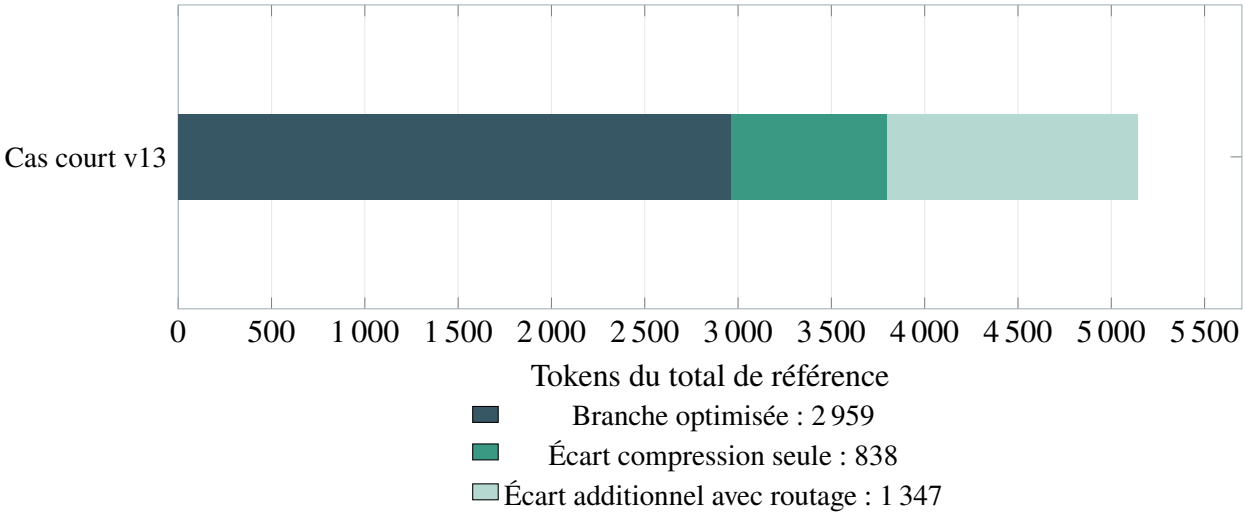


Figure 5.3 : Décomposition du total de 5 144 tokens dans l'expérience E3. L'économie totale est de 2 185 tokens, soit 42,48 %.

La durée de la branche optimisée est également inférieure dans cet essai : 44,3 secondes contre 60,9 pour la référence. Ce résultat accompagne l'économie de tokens sur cette exécution, mais ne permet pas de garantir une accélération identique sur une autre machine ou une autre charge du serveur.

5.3 Résultats répétés du protocole historique v12

Le protocole v12 a été exécuté deux fois sur chacune des trois sources ci-dessous. L'ordre des branches a été inversé à la deuxième répétition. Les six expériences appartiennent à la même campagne, sans suppression d'un résultat négatif ou d'un échec dans ce lot enregistré.

Source / répétition	T_B	T_C	T_A	R_A
Détaillé, 1	10 835	9 004	4 632	57,25 %
Détaillé, 2	10 835	9 007	4 632	57,25 %
Personnalisé court, 1	4 782	3 952	2 723	43,06 %
Personnalisé court, 2	4 784	3 952	2 723	43,08 %
Personnalisé long, 1	7 031	5 985	3 044	56,71 %
Personnalisé long, 2	7 046	5 985	3 044	56,80 %

Table 5.3 : Mesures v12 conservées pour l'analyse historique ; source E1.

Les dix-huit décisions ont été validées, ainsi que les six audits. Les 148 tuples factuels évalués étaient corrects. Une incertitude supplémentaire a été rejetée dans un rapport de référence parce qu'elle ne correspondait pas à une réserve source autorisée ; elle n'a pas été transmise. Aucun échec d'exécution, schéma invalide ou gain négatif n'a été observé dans ces six expériences. Leur coût physique cumulé est de 89 060 tokens.

La moyenne arithmétique des six réductions est de 52,36 %, avec un écart-type de 7,20 points de pourcentage selon la synthèse E1. Cette dispersion mélange des sources courtes et longues ; elle ne représente pas uniquement l'instabilité du modèle lorsqu'on répète une entrée identique.

Sur les deux sources longues de ce lot, le gain total dépasse 50 %, alors que la compression seule représente environ 14,88 à 16,90 %. Cette observation soutient l'intérêt de l'approche hybride dans les cas testés. Elle n'établit pas un minimum pour toute source longue. Deux répétitions ne permettent pas de caractériser finement la variance, et un échantillon favorable ne remplace pas des essais adverses ou des cas où la compression est inutile.

Séparation des versions

La revue explicite des exigences personnalisées a été ajoutée en v13. Les six expériences v12 ne sont pas renommées en v13 et ne prouvent pas rétroactivement les contrôles supplémentaires du nouveau protocole.

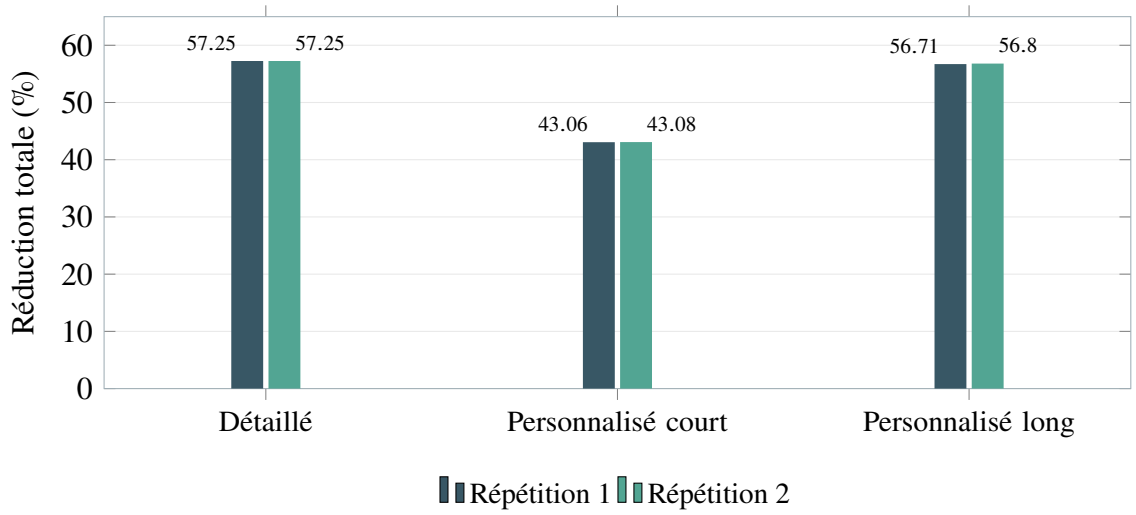


FIGURE 5.4 : Six observations historiques v12. Chaque barre correspond à une expérience enregistrée ; aucune moyenne interversions n'est utilisée.

5.4 Vérification des exigences en v13

Le vérificateur de besoins construit un scénario à six offres où le transport réfrigéré est obligatoire. D2 est admissible selon les quatre règles de base, mais ne possède pas la capacité déclarée nécessaire. D6 la possède. Les trois branches ont sélectionné D6 et satisfait 16 contrôles chacune.

Cas v13	T_B	T_C	T_A	Issue
Court standard	5 144	4 306	2 959	S1 ; 42,48 %
Réfrigération déclarée	12 016	10 180	5 232	D6 ; 56,46 %
Capacité inconnue pour D6	–	–	–	Revue ; 0 token LLM
Exigence précise de 2 à 8 °C	–	–	–	Revue ; 0 token LLM

TABLE 5.4 : Deux comparaisons v13 terminées et deux arrêts distincts ; sources E2 et E3.

Dans le cas réfrigéré, l'économie est de 6 784 tokens : 1 836 pour la comparaison compression seule et 4 948 additionnels avec routage. Les 6 527 tokens d'entrée et 257 tokens de sortie économisés donnent le même bilan total. Le coût physique des trois branches vaut 23 456 tokens.

Les deux cas arrêtés ne sont pas des décisions erronées présentées comme réussies. Ils montrent que le système s'abstient lorsque sa représentation ne permet pas le contrôle demandé. Le dashboard affiche alors *Not measured* pour les économies, car aucune comparaison d'inférence n'a eu lieu. Zéro token consommé avant décision ne signifie pas une économie de 100 % pour une tâche effectivement accomplie.

La synthèse E2 comprend trois tentatives : une comparaison terminée et deux arrêts. Elle compte 3 décisions validées sur 9 planifiées, mais 3 sur 3 parmi les décisions émises. Ces deux ratios répondent à des questions différentes. Le cas court E3 est une expérience distincte et ne doit pas être ajouté à cette synthèse sans déclarer le changement de périmètre.

5.5 Tests logiciels et contrôles d'interface

La suite logicielle complète a validé 147 tests lors de la vérification v13. Les cas couvrent notamment les comparaisons aux limites, la cohérence des faits, la structure des rapports, les contradictions, la décision finale, les sources personnalisées et les conditions d'arrêt. Ces tests contrôlés ne sont pas tous des appels réels à Llama 3 ; leur nombre ne doit pas être présenté comme un nombre de scénarios d'inférence indépendants.

Contrôle	Comportement attendu et vérifié
Budget, délai, capacité aux limites	Comparaisons exactes et cohérentes
Affirmation ou explication contradictoire	Rejet et conservation de l'original
Altération du contexte enregistré	Détection par reconstruction de l'audit
Exigence non prise en charge	Arrêt avant l'appel au modèle
Capacité fournisseur inconnue	Revue requise ; aucune assimilation à « oui »
Modification du formulaire	Confirmation effacée
Champs obligatoires repliés	Section révélée par la validation du formulaire

Table 5.5 : Exemples de garanties testées ; les propriétés sémantiques générales restent hors périmètre.

La simplification ultérieure de l'interface a été vérifiée au navigateur, et les 24 tests pertinents de revue, de scénario personnalisé et d'API ont été réexécutés avec succès. Le retrait des choix Legacy a également été contrôlé dans le sélecteur, sans suppression des anciens résultats de recherche.

5.6 Discussion et limites

5.6.1 Ce que les résultats démontrent

Les expériences établissent une réduction observée des tokens relativement à la référence de six rôles LLM. Elles montrent que les sorties acceptées respectent les règles calculées et que les prompts gardent la source dans le périmètre audité. Le contrôle d'une exigence supplémentaire peut modifier le choix, comme le passage de D2 à D6 lorsque la réfrigération est obligatoire.

5.6.2 Ce qu'elles ne démontrent pas

La référence est un workflow multi-agents existant, et non l'algorithme minimal possible. Une application purement Python pourrait sélectionner un devis avec encore moins d'appels LLM. Le présent protocole ne prouve donc pas l'optimalité globale de cette architecture, ni que chaque rôle conservé apporte une valeur indispensable.

La décision reçoit déjà la recommandation calculée. Son succès mesure une fidélité au calcul fourni, pas une découverte autonome de la solution. La conservation textuelle n'est pas une évaluation complète de la compréhension. Des conditions omises par l'utilisateur lors de la déclaration peuvent rester hors du contrôle formel.

Les règles de routage et de compression sont des heuristiques. Des scénarios peu répétitifs peuvent produire des gains plus faibles, une absence de compression ou une augmentation du coût global. Les traces étudiées ne permettent pas d'affirmer une économie universelle de 30 % ou de 50 %. Les durées observées sont dépendantes de la machine, du chargement du modèle et de la concurrence éventuelle sur le serveur local.

5.6.3 Améliorations évaluables

Un prolongement utile consisterait à constituer un corpus figé de scénarios variés avec annotation des exigences, y compris des cas ambigus et négatifs. Les versions seraient comparées sur ce même corpus, avec alternance des branches, davantage de répétitions et conservation de tous les échecs.

Une extraction assistée des exigences pourrait réduire la saisie manuelle, mais son coût et ses omissions devraient être mesurés. De nouvelles règles, par exemple des plages de température, nécessiteraient des données fournisseur adaptées et des tests explicites. Enfin, une comparaison avec une référence Python seule aiderait à quantifier la valeur ajoutée propre aux rôles LLM.

5.7 Conclusion

La réalisation fournit un démonstrateur local et des traces mesurables. La meilleure économie observée provient de la combinaison de la compression et du routage, ce que les trois branches permettent de distinguer. La validation reste bornée aux faits déclarés et aux propriétés effectivement contrôlées.

Conclusion générale et perspectives

Ce PFA a étudié la réduction du coût en tokens d'un workflow multi-agents au moyen d'une compression de contexte contrôlable et de fonctions Python. Le travail a progressivement déplacé les comparaisons factuelles vers une référence déterministe, isolé les rapports non validés et introduit une revue explicite des exigences personnalisées.

L'apport du prototype réside dans l'association de quatre éléments : un registre conservant la source, une projection compacte, un routage évitant certains appels spécialisés et une mesure transparente à trois branches. Les expériences montrent des économies dans les scénarios étudiés, sans attribuer l'ensemble du gain au compresseur. Elles rendent aussi visibles les conditions dans lesquelles le système doit s'arrêter pour revue.

La poursuite du projet devrait privilégier une évaluation plus large et une amélioration de la saisie des exigences plutôt qu'un objectif de pourcentage garanti. La généralisation à des documents libres nécessiterait une validation de leur interprétation, une analyse des omissions et un contrôle du coût de l'extraction. L'objectif demeure une réduction utile et mesurée des ressources, accompagnée d'une description précise de ce qui a été vérifié.

Annexe A

Traçabilité des mesures

Les fichiers ci-dessous sont conservés dans le dépôt du projet. Les chemins sont relatifs à sa racine. Il convient de les joindre ou de les archiver avec la version finale du rapport pour permettre l'examen des résultats.

E1 : campagne historique v12

`verification/adaptive-v12-20261001.json`

`verification/adaptive-v12-20261001.summary.json`

`verification/adaptive-v12-audit-20261001.json`

Six expériences ; trois sources, deux répétitions ; 18 décisions contrôlées.

E2 : exigences v13

`verification/requirements-v13-20261001.json`

`verification/requirements-v13-20261001.summary.json`

`verification/requirements-v13-20261001.audit.json`

Une comparaison réfrigérée terminée et deux arrêts avant inférence.

E3 : contrôle court par le dashboard

`verification/paired-f8fee089-1eb7-4622-962c-2891b0a1a2b7.json`

Expérience `f8fee089-1eb7-4622-962c-2891b0a1a2b7`, scénario `demo_short`.

E4 : compatibilité historique

`verification/requirements-compatibility-audit-20261001.json`

Réexamen des six traces v12 après l'ajout de la revue v13 : six audits conformes.

E5 : exécution illustrée dans les captures

`rapport/preuves/paired-52d148a8-9922-4a14-9545-7b1745333c92.json`

Expérience du 3 octobre 2026, scénario `demo_short`, protocole v13. Les totaux des trois branches sont respectivement 5 144, 4 306 et 2 959 tokens ; les durées affichées sont 53,3 s, 51,9 s et 39,1 s. Les trois décisions passent les quatorze contrôles. Cette trace documente les figures 4.2 à 4.5 ; elle n'est pas ajoutée aux séries historiques analysées au chapitre 5.

Les chemins et identifiants relient le texte aux mesures ; ils ne sont pas des preuves de provenance externes. Pour une remise académique reproductible, archiver aussi la révision du code,

l'environnement, le modèle exact et les résultats des tests. Aucun résultat du rapport fourni comme modèle de mise en page n'a été utilisé comme mesure de Token Efficient.

Annexe B

Reproduction et démonstration

B.1 Relevé de l'environnement lors de la revue finale

Le 3 octobre 2026, le fichier `venv/pyvenv.cfg` indique Python 3.14.7 et l'API locale d'Ollama annonce la version 0.35.0. Le modèle installé est `llama3:latest`, 8,0 milliards de paramètres, au format GGUF et quantifié en `Q4_0`. L'empreinte retournée par le serveur est :

`365c0bd3c000a25d28ddbf732fe1c6add414de7275464c4e4d1c3b5fcb5d8ad1`

Ce relevé, conservé dans `rapport/preuves/environnement-20261003.json`, décrit l'installation observée à cette date. Les anciennes traces n'ayant pas enregistré toutes ces informations, il ne constitue pas une preuve rétroactive des versions ou des poids utilisés pour chacune d'elles.

B.2 Préparation de l'environnement

Les commandes suivantes sont des repères pour Windows PowerShell à la racine du projet. Utiliser un environnement Python compatible avec les dépendances verrouillées et relever ses versions effectives. Ollama doit être installé et le modèle local disponible. Le démarrage exact peut être adapté à la machine.

```
python -m venv venv
.\venv\Scripts\python.exe -m pip install -r requirements.txt
ollama pull llama3
.\venv\Scripts\python.exe -m uvicorn api:app --host 127.0.0.1 --port 8000
```

Le dashboard se consulte sur `http://localhost:8000/`. Sur l'exemple court, ouvrir la source, lancer *Run comparison*, consulter S1 dans la décision, puis comparer les trois lignes de tokens et les contrôles. Une nouvelle exécution peut produire des longueurs de sortie différentes ; ne pas annoncer à l'avance le même pourcentage exact.

B.3 Tests et vérification

```
.\venv\Scripts\python.exe -B -m unittest discover -s tests
.\venv\Scripts\python.exe -B -u verify_requirements.py --output verification/
    requirements-new.json
.\venv\Scripts\python.exe -B context_integrity_audit.py verification/paired-f8fee089-1
    eb7-4622-962c-2891b0a1a2b7.json --output verification/audit-new.json
```

Choisir des fichiers de sortie nouveaux afin de conserver les preuves existantes. Le vérificateur d'exigences traite explicitement ses cas attendus de revue. Un audit de décision appliqué isolément à une expérience arrêtée avant inférence ne doit pas être interprété comme une décision validée.

B.4 Reproduire la campagne historique

```
.\venv\Scripts\python.exe -B -u evaluate_adaptive.py --protocol validated-v12-tools --
    case demo_detailed --custom-file verification/custom-adaptive-sources.json --repeats
     2 --seed 120 --output verification/adaptive-v12-new.json
```

Le paramètre de protocole est explicite car les anciens scénarios personnalisés ne possèdent pas nécessairement la revue requise par v13. Pour évaluer le protocole actuel, préparer des sources avec exigences confirmées et conserver son identifiant. Éviter de lancer simultanément d'autres tâches d'inférence lors d'une comparaison de latence.

Annexe C

Dictionnaire des résultats et statuts

Une expérience est enregistrée sous la forme d'un document JSON. Les champs ci-dessous permettent de relier les indicateurs du dashboard à leur origine, sans interpréter une absence de mesure comme un résultat nul.

Champ	Interprétation
`experiment_id`	Identifiant de l'expérience sauvegardée.
`scenario`	Source structurée : demande, contraintes et devis.
`configuration`	Protocole, modèle, graine, limites et ordre des branches.
`shared`	Travail d'ingestion exécuté une seule fois et partagé.
`requirement_validation`	État de la revue préalable et problèmes empêchant l'inférence.
`baseline`	Branche de référence avec contexte développé.
`compression_only`	Branche comprimée conservant les six rôles LLM.
`compressed`	Branche optimisée présentée dans l'interface.
`comparison.saved_tokens`	Différence entre les coûts de référence et optimisé.
`comparison.reduction_percent`	Pourcentage calculé à partir des totaux mesurés.
`comparison.compressor_tokens`	Coût d'inférence du compresseur ; nul pour la projection Python actuelle.
`comparison.total_experiment_tokens`	Coût physique de toute la comparaison, sans tripler l'ingestion partagée.
`context_audit`	Contrôles de reconstruction des faits, du contexte et des transmissions.
`claim_validation`	Affirmations évaluées, éléments rejetés et explication du filtrage.
`forwarded_output`	Texte autorisé à être utilisé par les étapes suivantes.

Table C.1 : Champs utiles à la lecture des traces d'expérience.

C.1 Lecture des issues

`validated`
: Les contrôles de l'expérience ont été satisfaits dans le périmètre défini. Ce statut n'atteste pas la vérité de données fournisseur inventées ou non déclarées.

`needs_review`
: Le scénario requiert une revue préalable. Aucune décision automatique n'est émise et les économies d'inférence ne sont pas mesurées.

`failed_validation`
: Une sortie ne respecte pas les contrôles attendus. La trace reste disponible et le cas ne doit pas être compté comme un succès de décision.

Compression ignorée
: La projection n'a pas été appliquée. Une différence éventuelle de longueur des sorties ne constitue pas une économie attribuable au compresseur.

C.2 Conservation des preuves

L'identifiant du protocole, les sources, les sorties brutes et les motifs de rejet doivent être conservés avec les compteurs. Une capture de l'interface facilite la présentation du résultat, mais le fichier JSON constitue la trace exploitable pour recalculer les différences et examiner les contrôles.

Bibliographie

[1] LangChain. *LangGraph overview*. Documentation officielle. https://docs.langchain.com/oss/python/langgraph/overview. Consultée le 2 octobre 2026.

[2] LangChain. *ChatOllama integration*. Documentation officielle. https://docs.langchain.com/oss/python/integrations/chat/ollama. Consultée le 2 octobre 2026.

[3] Ollama. *Generate a response : API reference*. Documentation officielle des réponses et compteurs. https://docs.ollama.com/api/generate. Consultée le 3 octobre 2026.

[4] FastAPI. *Documentation officielle du framework*. https://fastapi.tiangolo.com/. Consultée le 2 octobre 2026.

[5] Projet Token Efficient. *Code source local, protocoles v12/v13 et tests de validation*. État examiné au 2 octobre 2026. Composants présentés au chapitre 4.

[6] Projet Token Efficient. *Expériences et audits locaux du 1er octobre 2026*. Sources E1 à E4, annexe A.